Summarize

Chapter 5. Implementing Microservice Communication

As we discussed in the previous chapter, your choice of technology should be driven in large part by the style of communication you want. Deciding between blocking synchronous or nonblocking asynchronous calls, request-response or event-driven collaboration, will help you whittle down what might otherwise be a very long list of technology. In this chapter, we're going to look at some of the technology commonly used for microservice communication.

Looking for the Ideal Technology

There is a bewildering array of options for how one microservice can talk to another. But which is the right one—SOAP? XML-RPC? REST? gRPC? And new options are always coming out. So before we discuss specific technology, let's think about what we want out of which technology we pick.

Make Backward Compatibility Easy

When making changes to our microservices, we need to make sure we don't break compatibility with any consuming microservices. As such, we want to ensure that whatever technology we pick makes it easy to make backward-compatible changes. Simple operations like adding new fields shouldn't break clients. We also ideally want the ability to validate that the changes we have made are backward-compatible—and have a way to get that feedback before we deploy our microservice into production.

Make Your Interface Explicit

It is important that the interface that a microservice exposes to the outside world is explicit. This means that it is clear to a consumer of a microservice as to what functionality that microservice exposes. But it also means that it is clear to a developer working on

the microservice what functionality needs to remain intact for external parties—we want to avoid a situation in which a change to a microservice causes an accidental breakage in compatibility.

Explicit schemas can go a long way in helping ensure that the interface a microservice exposes is explicit. Some of the technology we can look at requires the use of a schema; for other technology, the use of a schema is optional. Either way, I strongly encourage the use of an explicit schema, as well as there being enough supporting documentation to be clear about what functionality a consumer can expect a microservice to provide.

Keep Your APIs Technology Agnostic

If you have been in the IT industry for more than 15 minutes, you don't need me to tell you that we work in a space that is changing rapidly. The one certainty *is* change. New tools, frameworks, and languages are coming out all the time, implementing new ideas that can help us work faster and more effectively. Right now, you might be a .NET shop. But what about a year from now, or five years from now? What if you want to experiment with an alternative technology stack that might make you more productive?

I am a big fan of keeping my options open, which is why I am such a fan of microservices. It is also why I think it is very important to ensure that you keep the APIs used for communication between microservices technology agnostic. This means avoiding integration technology that dictates what technology stacks we can use to implement our microservices.

Make Your Service Simple for Consumers

We want to make it easy for consumers to use our microservice. Having a beautifully factored microservice doesn't count for much if the cost of using it as a consumer is sky high! So let's think about what makes it easy for consumers to use our wonderful new service. Ideally, we'd like to allow our clients full freedom in their technology choice; on the other hand, providing a client library can ease adoption. Often, however, such libraries are incompatible with other things we want to achieve. For example, we might use client libraries to make it easy for consumers, but this can come at the cost of increased coupling.

Hide Internal Implementation Detail

We don't want our consumers to be bound to our internal implementation, as it leads to increased coupling; that in turn means that if we want to change something inside our microservice, we can break our consumers by requiring them to also change. That increases the cost of change—exactly what we are trying to avoid. It also means we are less likely to want to make a change for fear of having to upgrade our consumers, which can lead to increased technical debt within the service. So any technology that pushes us to expose internal representation detail should be avoided.

Technology Choices

There is a whole host of technology we could look at, but rather than looking broadly at a long list of options, I will highlight some of the most popular and interesting choices. Here are the options we'll be looking at:

Remote procedure calls

: Frameworks that allow for local method calls to be invoked on a remote process. Common options include SOAP and gRPC.

REST

: An architectural style where you expose resources (Customer, Order, etc.) that can be accessed using a common set of verbs (GET, POST). There is a bit more to REST than this, but we'll get to that shortly.

GraphQL

: A relatively new protocol that allows consumers to define custom queries that can fetch information from multiple downstream microservices, filtering the results to return only what is needed.

Message brokers

: Middleware that allows for asynchronous communication via queues or topics.

Remote Procedure Calls

Remote procedure call (RPC) refers to the technique of making a local call and having it execute on a remote service somewhere. There are a number of different RPC implementations in use. Most of the technology in this space requires an explicit schema, such as

SOAP or gRPC. In the context of RPC, the schema is often referred to as an interface definition language (IDL), with SOAP referring to its schema format as a web service definition language (WSDL). The use of a separate schema makes it easier to generate client and server stubs for different technology stacks—so, for example, I could have a Java server exposing a SOAP interface, and a .NET client generated from the same WSDL definition of the interface. Other technology, such as Java RMI, calls for a tighter coupling between the client and the server, requiring that both use the same underlying technology but avoid the need for an explicit service definition, as the service definition is implicitly provided by the Java type definitions. All these technologies, however, have the same core characteristic: they make a remote call look like a local call.

Typically, using an RPC technology means you are buying into a serialization protocol. The RPC framework defines how data is serialized and deserialized. For instance, gRPC uses the protocol buffer serialization format for this purpose. Some implementations are tied to a specific networking protocol (like SOAP, which makes nominal use of HTTP), whereas others might allow you to use different types of networking protocols, which can provide additional features. For example, TCP offers guarantees about delivery, whereas UDP doesn't but has a much lower overhead. This can allow you to use different networking technology for different use cases.

RPC frameworks that have an explicit schema make it very easy to generate client code. This can avoid the need for client libraries, as any client can just generate their own code against this service specification. For client-side code generation to work, though, the client needs some way to get the schema out of band—in other words, the consumer needs to have access to the schema before it plans to make calls. Avro RPC is an interesting outlier here, as it has the option to send the full schema along with the payload, allowing clients to dynamically interpret the schema.

The ease of generation of client-side code is one of the main selling points of RPC. The fact that I can just make a normal method call and theoretically ignore the rest is a huge boon.

Challenges

As we've seen, RPC offers some great advantages, but it's not without its downsides—and some RPC implementations can be more problematic than others. Many of these issues can be dealt with, but they deserve further exploration.

Technology coupling

Some RPC mechanisms, like Java RMI, are heavily tied to a specific platform, which can limit which technology can be used in the client and server. Thrift and gRPC have an impressive amount of support for alternative languages, which can reduce this downside somewhat, but be aware that RPC technology sometimes comes with restrictions on interoperability.

In a way, this technology coupling can be a form of exposing internal technical implementation details. For example, the use of RMI ties not only the client to the JVM but the server as well.

To be fair, there are a number of RPC implementations that don't have this restriction—gRPC, SOAP, and Thrift are all examples that allow for interoperability between different technology stacks.

Local calls are not like remote calls

The core idea of RPC is to hide the complexity of a remote call. However, this can lead to hiding too much. The drive in some forms of RPC to make remote method calls look like local method calls hides the fact that these two things are very different. I can make large numbers of local, in-process calls without worrying too much about the performance. With RPC, though, the cost of marshaling and unmarshaling payloads can be significant, not to mention the time taken to send things over the network. This means you need to think differently about API design for remote interfaces versus local interfaces. Just taking a local API and trying to make it a service boundary without any more thought is likely to get you in trouble. In some of the worst examples, developers may be using remote calls without knowing it, if the abstraction is overly opaque.

You need to think about the network itself. Famously, the first of the fallacies of distributed computing is "The network is reliable". Networks *aren't* reliable. They can and will fail, even if your client and the server you are speaking to are fine. They can fail fast, they can fail slow, and they can even malform your packets. You should assume that your networks are plagued with malevolent entities ready to unleash their ire on a whim. Therefore, you can expect to encounter types of failure modes that you may never have had to deal with in simpler, monolithic software. A failure could be caused by the remote server returning an error, or by you making a bad call. Can you tell the difference, and if so, can you do anything about it? And what do you do when the remote server just starts responding slowly? We'll cover this topic when we talk about resiliency in Chapter 12.

Brittleness

Some of the most popular implementations of RPC can lead to some nasty forms of brittleness, Java RMI being a very good example. Let's consider a very simple Java interface that we have decided to make a remote API for our `Customer` service. Example 5-1 declares the methods we are going to expose remotely. Java RMI then generates the client and server stubs for our method.

Example 5-1. Defining a service endpoint using Java RMI

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface CustomerRemote extends Remote {
  public Customer findCustomer(String id) throws RemoteException;

  public Customer createCustomer(
    String firstname, String surname, String emailAddress)
    throws RemoteException;
}
```

In this interface, `createCustomer` takes the first name, surname, and email address. What happens if we decide to allow the `Customer` object to also be created with just an email address? We could add a new method at this point pretty easily, like so:

```
...
public Customer createCustomer(String emailAddress) throws RemoteException;
...
```

The problem is that now we need to regenerate the client stubs too. Clients that want to consume the new method need the new stubs, and depending on the nature of the changes to the specification, consumers that don't need the new method may also need to have their stubs upgraded. This is manageable, of course, but only to a point. The reality is that changes like this are fairly common. RPC endpoints often end up having a large number of methods for different ways of creating or interacting with objects. This is due in part to the fact that we are still thinking of these remote calls as local ones.

There is another sort of brittleness, though. Let's take a look at what our `Customer` object looks like:

```
public class Customer implements Serializable {
  private String firstName;
  private String surname;
  private String emailAddress;
  private String age;
}
```

What if it turns out that, although we expose the `age` field in our `Customer` objects, none of our consumers ever use it? We decide we want to remove this field. But if the server implementation removes `age` from its definition of this type, and we don't do the same to all the consumers, then even though they never used the field, the code associated with deserializing the `Customer` object on the consumer side will break. To roll out this change, we'd need to make changes to the client code to support the new definition and deploy these updated clients at the same time as we roll out the new version of the server. This is a key challenge with any RPC mechanism that promotes the use of binary stub generation: you don't get to separate client and server deployments. If you use this technology, lockstep releases may be in your future.

Similar problems occur if we want to restructure the `Customer` object, even if we didn't remove fields—for example, if we wanted to encapsulate `firstName` and `surname` into a new `naming` type to make it easier to manage. We could, of course, fix this by passing around dictionary types as the parameters of our calls, but at that point, we lose many of the benefits of the generated stubs because we'll still have to manually match and extract the fields we want.

In practice, objects used as part of binary serialization across the wire can be thought of as "expand-only" types. This brittleness results in the types being exposed over the wire and becoming a mass of fields, some of which are no longer used but can't be safely removed.

Where to use it

Despite its shortcomings, I actually quite like RPC, and the more modern implementations, such as gRPC, are excellent, whereas other implementations have significant issues that would cause me to give them a wide berth. Java RMI, for example, has a number of issues regarding brittleness and limited technology choices, and SOAP is pretty heavyweight from a developer perspective, especially when compared with more modern choices.

Just be aware of some of the potential pitfalls associated with RPC if you're going to pick this model. Don't abstract your remote calls to the point that the network is completely hidden, and ensure that you can evolve the server interface without having to insist on lockstep upgrades for clients. Finding the right balance for your client code is important, for example. Make sure your clients aren't oblivious to the fact that a network call is going to be made. Client libraries are often used in the context of RPC, and if not structured right they can be problematic. We'll talk more about them shortly.

If I was looking at options in this space, gRPC would be at the top of my list. Built to take advantage of HTTP/2, it has some impressive performance characteristics and good general ease of use. I also appreciate the ecosystem around gRPC, including tools like Protolock, which is something we'll discuss later in this chapter when we discuss schemas.

gRPC fits a synchronous request-response model well but can also work in conjunction with reactive extensions. It's high on my list whenever I'm in situations where I have a good deal of control over both the client and server ends of the spectrum. If you're having to support a wide variety of other applications that might need to talk to your microservices, the need to compile client-side code against a server-side schema can be problematic. In that case, some form of REST over HTTP API would likely be a better fit.

REST

Representational State Transfer (REST) is an architectural style inspired by the web. There are many principles and constraints behind the REST style, but we are going to focus on those that really help us when we face integration challenges in a microservices world, and when we're looking for an alternative to RPC for our service interfaces.

Most important when thinking about REST is the concept of resources. You can think of a resource as a thing that the service itself knows about, like a `Customer`. The server creates different representations of this `Customer` on request. How a resource is shown externally is completely decoupled from how it is stored internally. A client might ask for a JSON representation of a `Customer`, for example, even if it is stored in a completely different format. Once a client has a representation of this `Customer`, it can then make requests to change it, and the server may or may not comply with them.

There are many different styles of REST, and I touch only briefly on them here. I strongly recommend you take a look at the Richardson Maturity Model, where the different styles of REST are compared.

REST itself doesn't really talk about underlying protocols, although it is most commonly used over HTTP. I have seen implementations of REST using very different protocols before, although this can require a lot of work. Some of the features that HTTP gives us as part of the specification, such as verbs, make implementing REST over HTTP easier, whereas with other protocols you'll have to handle these features yourself.

REST and HTTP

HTTP itself defines some useful capabilities that play very well with the REST style. For instance, the HTTP verbs (e.g., GET, POST, and PUT) already have well-understood meanings in the HTTP specification as to how they should work with resources. The REST architectural style actually tells us that these verbs should behave the same way on all resources, and the HTTP specification happens to define a bunch of verbs we can use. GET retrieves a resource in an idempotent way, for example, and POST creates a new resource. This means we can avoid lots of different `createCustomer` or `editCustomer` methods. Instead, we can simply POST a customer representation to request that the server create a new resource, and then we can initiate a GET request to retrieve a representation of a resource. Conceptually, there is one *endpoint* in the form of a `Customer` resource in these cases, and the operations we can carry out on it are baked into the HTTP protocol.

HTTP also brings a large ecosystem of supporting tools and technology. We get to use HTTP caching proxies like Varnish and load balancers like `mod_proxy`, and many monitoring tools already have lots of support for HTTP out of the box. These building blocks allow us to handle large volumes of HTTP traffic and route them smartly, and in a fairly transparent way. We also get to use all the available security controls with HTTP to secure our communications. From basic auth to client certs, the HTTP ecosystem gives us lots of tools to make the security process easier, and we'll explore that topic more in Chapter 11. That said, to get these benefits, you have to use HTTP well. Use it badly, and it can be as insecure and hard to scale as any other technology out there. Use it right, though, and you get a lot of help.

Note that HTTP can be used to implement RPC too. SOAP, for example, gets routed over HTTP, but it unfortunately uses very little of the specification. Verbs are ignored, as are simple things like HTTP error codes. On the other hand, gRPC has been designed to take advantage of the capabilities of HTTP/2, such as the ability to send multiple request-response streams over a single connection. But of course, when using gRPC, you're not doing REST just because you're using HTTP!

Hypermedia as the engine of application state

Another principle introduced in REST that can help us avoid the coupling between client and server is the concept of *hypermedia as the engine of application state* (often abbreviated as HATEOAS, and boy, did it need an abbreviation). This is fairly dense wording and a fairly interesting concept, so let's break it down a bit.

Hypermedia is a concept wherein a piece of content contains links to various other pieces of content in a variety of formats (e.g., text, images, sounds). This should be pretty familiar to you, as it's what happens in the average web page: you follow links, which are a form of hypermedia controls, to see related content. The idea behind HATEOAS is that clients should perform interactions with the server (potentially leading to state transitions) via these links to other resources. A client doesn't need to know where exactly customers live on the server by knowing which URI to hit; instead, the client looks for and navigates links to find what it needs.

This is a bit of an odd concept, so let's first step back and consider how people interact with a web page, which we have already established is rich with hypermedia controls.

Think of the Amazon.com shopping site. The location of the shopping cart has changed over time. The graphic has changed. The link has changed. But as humans we are smart enough to still see a shopping cart, know what it is, and interact with it. We have an understanding of what a shopping cart means, even if the exact form and underlying control used to represent it have changed. We know that if we want to view the cart, this is the control we want to interact with. This is how web pages can change incrementally over time. As long as these implicit contracts between the customer and the website are still met, changes don't need to be breaking changes.

With hypermedia controls, we are trying to achieve the same level of "smarts" for our electronic consumers. Let's look at a hypermedia control that we might have for MusicCorp. We've accessed a resource representing a catalog entry for a given album in Example 5-2. Along with information about the album, we see a number of hypermedia controls.

Example 5-2. Hypermedia controls used on an album listing

```
<album>
  <name>Give Blood</name>
  <link rel="/artist" href="/artist/theBrakes" /> ❶
  <description>
    Awesome, short, brutish, funny and loud. Must buy!
```

```
  </description>
  <link rel="/instantpurchase" href="/instantPurchase/1234" /> ❷
</album>
```

❶ This hypermedia control shows us where to find information about the artist.

❷ And if we want to purchase the album, we now know where to go.

In this document, we have two hypermedia controls. The client reading such a document needs to know that a control with a relation of `artist` is where it needs to navigate to get information about the artist, and that `instantpurchase` is part of the protocol used to purchase the album. The client has to understand the semantics of the API in much the same way that a human needs to understand that on a shopping website the cart is where the items to be purchased will be.

As a client, I don't need to know which URI scheme to access to *buy* the album; I just need to access the resource, find the buy control, and navigate to that. The buy control could change location, the URI could change, or the site could even send me to another service altogether, and as a client I wouldn't care. This gives us a huge amount of decoupling between the client and the server.

We are greatly abstracted from the underlying detail here. We could completely change the implementation of how the control is presented as long as the client can still find a control that matches its understanding of the protocol, in the same way that a shopping cart control might go from being a simple link to a more complex JavaScript control. We are also free to add new controls to the document, perhaps representing new state transitions that we can perform on the resource in question. We would end up breaking our consumers only if we fundamentally changed the semantics of one of the controls so it behaved very differently, or if we removed a control altogether.

The theory is that, by using these controls to decouple the client and server, we gain significant benefits over time that hopefully offset the increase in the time it takes to get these protocols up and running. Unfortunately, although these ideas all seem sensible in theory, I've found that this form of REST is rarely practiced, for reasons I've not entirely come to grips with. This makes HATEOAS in particular a much harder concept for me to promote for those already committed to the use of REST. Fundamentally, many of the ideas in REST are predicated on creating distributed hypermedia systems, and this isn't what most people end up building.

Challenges

In terms of ease of consumption, historically you wouldn't be able to generate client-side code for your REST over HTTP application protocol like you can with RPC implementations. This has often lead to people creating REST APIs that provide client libraries for consumers to make use of. These client libraries give you a binding to the API to make client integration easier. The problem is that client libraries can cause some challenges with regards to coupling between the client and the server, something we'll discuss in "DRY and the Perils of Code Reuse in a Microservice World".

In recent years this problem has been somewhat alleviated. The OpenAPI specification that grew out of the Swagger project now provides you with the ability to define enough information on a REST endpoint to allow for the generation of client-side code in a variety of languages. In my experience, I haven't seen many teams actually making use of this functionality, even if they were already using Swagger for documentation. I have a suspicion that this may be due to the difficulties of retrofitting its use into current APIs. I do also have concerns about a specification that was previously used just for documentation now being used to define a more explicit contract. This can lead to a much more complex specification—comparing an OpenAPI schema with a protocol buffer schema, for example, is quite a stark contrast. Despite my reservations, though, it's good that this option now exists.

Performance may also be an issue. REST over HTTP payloads can actually be more compact than SOAP because REST supports alternative formats like JSON or even binary, but it will still be nowhere near as lean a binary protocol as Thrift might be. The overhead of HTTP for each request may also be a concern for low-latency requirements. All mainstream HTTP protocols in current use require the use of the Transmission Control Protocol (TCP) under the hood, which has inefficiencies compared with other networking protocols, and some RPC implementations allow you to use alternative networking protocols to TCP such as the User Datagram Protocol (UDP).

The limitations placed on HTTP due to the requirement to use TCP are being addressed. HTTP/3, which is currently in the process of being finalized, is looking to shift over to using the newer QUIC protocol. QUIC provides the same sorts of capabilities as TCP (such as improved guarantees over UDP) but with some significant enhancements that have been shown to deliver improvements in latency and reductions in bandwidth. It's likely to be several years before HTTP/3 has a widespread impact on the public internet, but it seems reasonable to assume that organizations can benefit earlier than this within their own networks.

With respect to HATEOAS specifically, you can encounter additional performance issues. As clients need to navigate multiple controls to find the right endpoints for a given operation, this can lead to very chatty protocols—multiple round trips may be required for each operation. Ultimately, this is a trade-off. If you decide to adopt a HATEOAS-style of REST, I would suggest you start by having your clients navigate these controls first and then optimize later if necessary. Remember that using HTTP provides us with a large amount of help out of the box, which we discussed earlier. The evils of premature optimization have been well documented before, so I don't need to expand on them here. Also note that a lot of these approaches were developed to create distributed hypertext systems, and not all of them fit! Sometimes you'll find yourself just wanting good old-fashioned RPC.

Despite these disadvantages, REST over HTTP is a sensible default choice for service-to-service interactions. If you want to know more, I recommend *REST in Practice: Hypermedia and Systems Architecture* (O'Reilly) by Jim Webber, Savas Parastatidis, and Ian Robinson, which covers the topic of REST over HTTP in depth.

Where to use it

Due to its widespread use in the industry, a REST-over-HTTP-based API is an obvious choice for a synchronous request-response interface if you are looking to allow access from as wide a variety of clients as possible. It would be a mistake to think of a REST API as just being a "good enough for most things" choice, but there is something to that. It's a widely understood style of interface that most people are familiar with, and it guarantees interoperability from a huge variety of technologies.

Due in large part to the capabilities of HTTP and the extent to which REST builds on these capabilities (rather than hiding them), REST-based APIs excel in situations in which you want large-scale and effective caching of requests. It's for this reason that they are the obvious choice for exposing APIs to external parties or client interfaces. They may well suffer, though, when compared to more efficient communication protocols, and although you can construct asynchronous interaction protocols over the top of REST-based APIs, that's not really a great fit compared to the alternatives for general microservice-to-microservice communication.

Despite intellectually appreciating the goals behind HATEOAS, I haven't seen much evidence that the additional work to implement this style of REST delivers worthwhile benefits in the long run, nor can I recall in the last few years talking to any teams implementing a microservice architecture that can speak to the value of using HATEOAS. My own experiences are obviously only one set of data points, and I don't doubt that for

some people HATEOAS may have worked well. But this concept does not seem to have caught on as much as I thought it would. It could be that the concepts behind HATEOAS are too alien for us to grasp, or it could be the lack of tools or standards in this space, or perhaps the model just doesn't work for the sorts of systems we have ended up building. It's also possible, of course, that the concepts behind HATEOAS don't really mix well with how we build microservices.

So for use at the perimeter, it works fantastically well, and for synchronous request-response-based communication between microservices, it's great.

GraphQL

In recent years, GraphQL has gained more popularity, due in large part to the fact that it excels in one specific area. Namely, it makes it possible for a client-side device to define queries that can avoid the need to make multiple requests to retrieve the same information. This can offer significant improvements in terms of the performance of constrained client-side devices and can also avoid the need to implement bespoke server-side aggregation.

To take a simple example, imagine a mobile device that wants to display a page showing an overview of a customer's latest orders. The page needs to contain some information about the customer, along with information about the customer's five most recent orders. The screen needs only a few fields from the customer record, and only the date, value, and shipped status of each order. The mobile device could issue calls to two downstream microservices to retrieve the required information, but this would involve making multiple calls, including pulling back information that isn't actually required. Especially with mobile devices, this can be wasteful—it uses up more of a mobile device's data plan than is needed, and it can take longer.

GraphQL allows the mobile device to issue a single query that can pull back all the required information. For this to work, you need a microservice that exposes a GraphQL endpoint to the client device. This GraphQL endpoint is the entry for all client queries and exposes a schema for the client devices to use. This schema exposes the types available to the client, and a nice graphical query builder is also available to make creating these queries easier. By reducing the number of calls and the amount of data retrieved by the client device, you can deal neatly with some of the challenges that occur when building user interfaces with microservice architectures.

Challenges

Early on, one challenge was lack of language support for the GraphQL specification, with JavaScript being your only choice initially. This has improved greatly, with all major technologies now having support for the specification. In fact, there have been significant improvements in GraphQL and the various implementations across the board, making GraphQL a much less risky prospect than it might have been a few years ago. That said, you might want to be aware of a few remaining challenges with the technology.

For one, the client device can issue dynamically changing queries, and I've heard of teams who have had issues with GraphQL queries causing significant load on the server side as a result of this capability. When we compare GraphQL with something like SQL, we see a similar issue. An expensive SQL statement can cause significant problems for a database and potentially have a large impact on the wider system. The same problem applies with GraphQL. The difference is that with SQL we at least have tools like query planners for our databases, which can help us diagnose problematic queries, whereas a similar problem with GraphQL can be harder to track down. Server-side throttling of requests is one potential solution, but as the execution of the call may be spread across multiple microservices, this is far from straightforward.

Compared with normal REST-based HTTP APIs, caching is also more complex. With REST-based APIs, I can set one of many response headers to help client-side devices, or intermediate caches like content delivery networks (CDNs), cache responses so they don't need to be requested again. This isn't possible in the same way with GraphQL. The advice I've seen on this issue seems to revolve around just associating an ID with every returned resource (and remember, a GraphQL query could contain multiple resources) and then having the client device cache the request against that ID. As far as I can tell, this makes using CDNs or caching reverse proxies incredibly difficult without additional work.

Although I've seen some implementation-specific solutions to this problem (such as those found in the JavaScript Apollo implementation), caching feels like it was either consciously or unconsciously ignored as part of the initial development of GraphQL. If the queries you are issuing are highly specific in nature to a particular user, then this lack of request-level caching may not be a deal breaker, of course, as your cache-hit ratio is likely to be low. I do wonder, though, if this limitation means that you'll still end up with a hybrid solution for client devices, with some (more generic) requests going over normal REST-based HTTP APIs and other requests going over GraphQL.

Another issue is that while GraphQL theoretically can handle writes, it doesn't seem to fit as well as for reads. This leads to situations in which teams are using GraphQL for read but REST for writes.

The last issue is something that may be entirely subjective, but I still think it's worth raising. GraphQL makes it feel like you are just working with data, which can reinforce the idea that the microservices you are talking to are just wrappers over databases. In fact, I've seen multiple people compare GraphQL to OData, a technology that is designed as a generic API for accessing data from databases. As we've already discussed at length, the idea of treating microservices just as wrappers over databases can be very problematic. Microservices expose functionality over networked interfaces. Some of that functionality might require or result in data being exposed, but they should still have their own internal logic and behavior. Just because you are using GraphQL, don't slip into thinking of your microservices as little more than an API on a database—it's essential that your GraphQL API isn't coupled to the underlying datastores of your microservices.

Where to use it

GraphQL's sweet spot is for use at the perimeter of the system, exposing functionality to external clients. These clients are typically GUIs, and it's an obvious fit for mobile devices given their constraints in terms of their limited ability to surface data to the end user and the nature of mobile networks. But GraphQL has also seen use for external APIs, GitHub being an early adopter of GraphQL. If you have an external API that often requires external clients to make multiple calls to get the information they need, then GraphQL can help make the API much more efficient and friendly.

Fundamentally, GraphQL is a call aggregation and filtering mechanism, so in the context of a microservice architecture it would be used to aggregate calls over multiple downstream microservices. As such, it's not something that would replace general microservice-to-microservice communication.

An alternative to the use of GraphQL would be to consider an alternative pattern like the backend for frontend (BFF) pattern—we'll look at that and compare it with GraphQL and other aggregation techniques in Chapter 14.

Message Brokers

Message brokers are intermediaries, often called middleware, that sit between processes to manage communication between them. They are a popular choice to help implement

asynchronous communication between microservices, as they offer a variety of powerful capabilities.

As we discussed earlier, a message is a generic concept that defines the thing that a message broker sends. A message could contain a request, a response, or an event. Rather than one microservice directly communicating with another microservice, the microservice instead gives a message to a message broker, with information about how the message should be sent.

Topics and queues

Brokers tend to provide either queues or topics, or both. Queues are typically point to point. A sender puts a message on a queue, and a consumer reads from that queue. With a topic-based system, multiple consumers are able to subscribe to a topic, and each subscribed consumer will receive a copy of that message.

A consumer could represent one or more microservices—typically modeled as a consumer group. This would be useful when you have multiple instances of a microservice, and you want any one of them to be able to receive a message. In Figure 5-1, we see an example in which the `Order Processor` has three deployed instances, all as part of the same consumer group. When a message is put into the queue, only one member of the consumer group will receive that message; this means the queue works as a load distribution mechanism. This is an example of the competing consumers pattern we touched on briefly in Chapter 4.

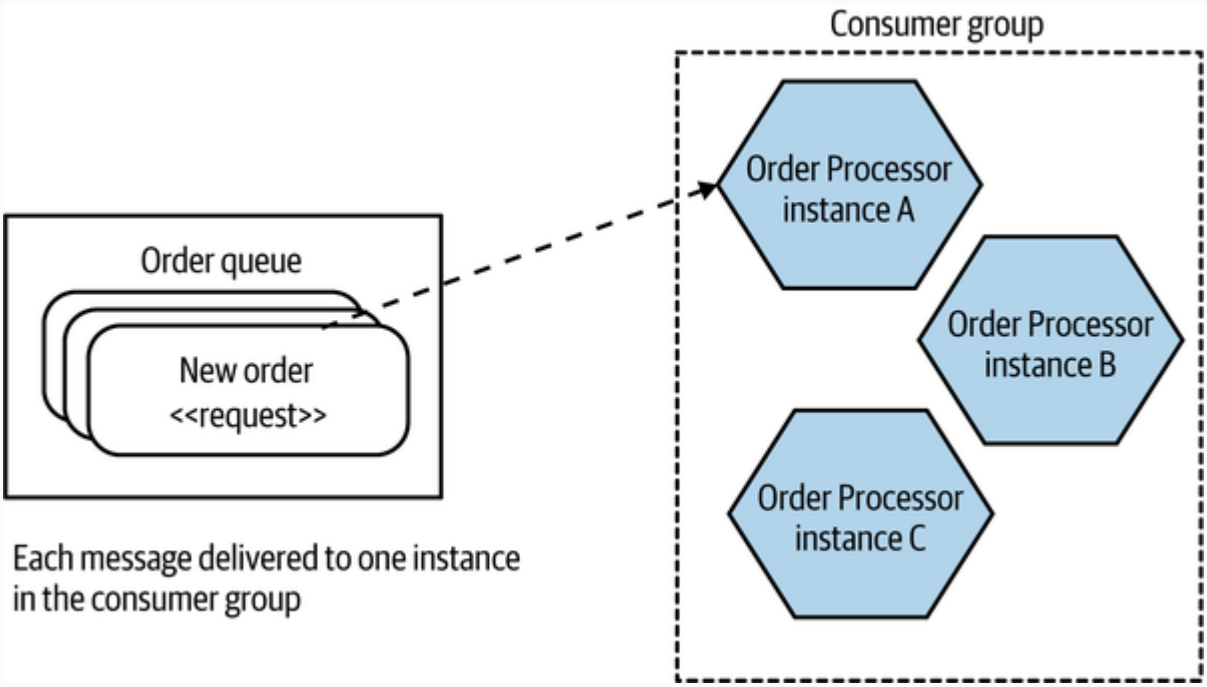


Figure 5-1. A queue allows for one consumer group

With topics, you can have multiple consumer groups. In Figure 5-2, an event representing an order being paid for is put onto the `Order Status` topic. A copy of that event is received by both the `Warehouse` microservice and the `Notifications` microservice, which are in separate consumer groups. Only one instance of each consumer group will see that event.

At first glance, a queue just looks like a topic with a single consumer group. A large part of the distinction between the two is that when a message is sent over a queue, there is knowledge of what the message is being sent to. With a topic, this information is hidden from the sender of the message—the sender is unaware of who (if anyone) will end up receiving the message.

Topics are a good fit for event-based collaboration, whereas queues would be more appropriate for request/response communication. This should be considered as general guidance rather than a strict rule, however.

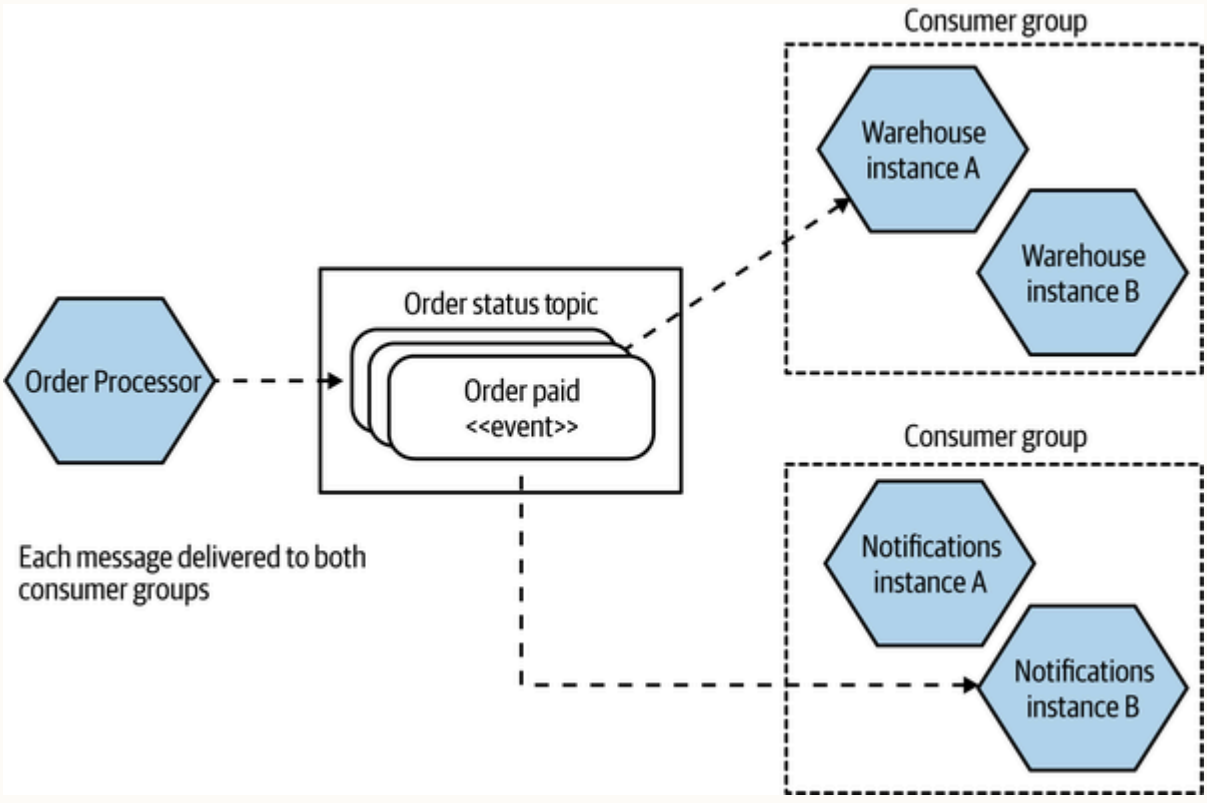


Figure 5-2. Topics allow for multiple subscribers to receive the same messages, which is useful for event broadcast

Guaranteed delivery

So why use a broker? Fundamentally, they provide some capabilities that can be very useful for asynchronous communication. The properties they provide vary, but the most interesting feature is that of guaranteed delivery, something that all widely used brokers support in some way. Guaranteed delivery describes a commitment by the broker to ensure that the message is delivered.

From the point of view of the microservice sending the message, this can be very useful. It's not a problem if the downstream destination is unavailable—the broker will hold on to the message until it can be delivered. This can reduce the number of things an upstream microservice needs to worry about. Compare that to a synchronous direct call—for example, an HTTP request: if the downstream destination isn't reachable, the upstream microservice will need to work out what to do with the request; should it retry the call or give up?

For guaranteed delivery to work, a broker will need to ensure that any messages not yet delivered are going to be held in a durable fashion until they can be delivered. To deliver on this promise, a broker will normally run as some sort of cluster-based system, ensuring that the loss of a single machine doesn't cause a message to be lost. There is typically a lot involved in running a broker correctly, partly due to the challenges in managing cluster-based software. Often, the promise of guaranteed delivery can be undermined if the broker isn't set up correctly. As an example, RabbitMQ requires instances in a cluster to communicate over relatively low-latency networks; otherwise the instances can start to get confused about the current state of messages being handled, resulting in data loss. I'm not highlighting this particular limitation as a way of saying that RabbitMQ is in any way bad—all brokers have restrictions as to how they need to be run to deliver the promise of guaranteed delivery. If you plan to run your own broker, make sure you read the documentation carefully.

It's also worth noting that what any given broker means by guaranteed delivery can vary. Again, reading the documentation is a great start.

Trust

One of the big draws of a broker is the property of guaranteed delivery. But for this to work, you need to trust not only the people who created the broker but also the way that broker has operated. If you've built a system that is based on the assumption that delivery is guaranteed, and that turns out not to be the case due to an issue with the underlying broker, it can cause significant issues. The hope, of course, is that you are offloading that work to software created by people who can do that job better than you can. Ultimately, you have to decide how much you want to trust the broker you are making use of.

Other characteristics

Besides guaranteed delivery, brokers can provide other characteristics that you may find useful.

Most brokers can guarantee the order in which messages will be delivered, but this isn't universal, and even then the scope of this guarantee can be limited. With Kafka, for example, ordering is guaranteed only within a single partition. If you can't be certain that messages will be received in order, your consumer may need to compensate, perhaps by deferring processing of messages that are received out of order until any missing messages are received.

Some brokers provide transactions on write—for instance, Kafka allows you to write to multiple topics in a single transaction. Some brokers can also provide read transactionality, something I've taken advantage of when using a number of brokers via the Java Message Service (JMS) APIs. This can be useful if you want to ensure the message can be processed by the consumer before removing it from the broker.

Another, somewhat controversial feature promised by some brokers is that of exactly once delivery. One of the easier ways to provide guaranteed delivery is allowing the message to be resent. This can result in a consumer seeing the same message more than once (even if this is a rare situation). Most brokers will do what they can to reduce the chance of this, or hide this fact from the consumer, but some brokers go further by guaranteeing exactly once delivery. This is a complex topic, as I've spoken to some experts who state that guaranteeing exactly once delivery in all cases is impossible, while other experts say you basically can do so with a few simple workarounds. Either way, if your broker of choice claims to implement this, then pay *really* careful attention to how it is implemented. Even better, build your consumers in such a way that they are prepared for the fact that they might receive a message more than once and can handle this situation. A very simple example would be for each message to have an ID, which a consumer can check each time a message is received. If a message with that ID has already been processed, the newer message can be ignored.

Choices

A variety of message brokers exist. Popular examples include RabbitMQ, ActiveMQ, and Kafka (which we'll explore further shortly). The main public cloud vendors also provide a variety of products that play this role, from managed versions of those brokers that you can install on your own infrastructure to bespoke implementations that are specific to a given platform. AWS, for example, has Simple Queue Service (SQS), Simple Notification Service (SNS), and Kinesis, all of which provide different flavors of fully managed brokers. SQS was in fact the second-ever product released by AWS, having been launched back in 2006.

Kafka

Kafka is worth highlighting as a specific broker, due in large part to its recent popularity. Part of that popularity is due to Kafka's use in helping move large volumes of data around as part of implementing stream processing pipelines. This can help move from batch-oriented processing to more real-time processing.

There are a few characteristics of Kafka that are worth highlighting. Firstly, it is designed for very large scale—it was built at LinkedIn to replace multiple existing message clusters with a single platform. Kafka is built to allow for multiple consumers and producers—I've spoken to one expert at a large technology company who had over fifty thousand producers and consumers working on the same cluster. To be fair, very few organizations have problems at that level of scale, but for some organizations, the ability to scale Kafka easily (relatively speaking) can be very useful.

Another fairly unique feature of Kafka is message permanence. With a normal message broker, once the last consumer has received a message, the broker no longer needs to hold on to that message. With Kafka, messages can be stored for a configurable period. This means that messages can be stored forever. This can allow consumers to reingest messages that they had already processed, or allow newly deployed consumers to process messages that were sent previously.

Finally, Kafka has been rolling out built-in support for stream processing. Rather than using Kafka to send messages to a dedicated stream processing tool like Apache Flink, some tasks can instead be done inside Kafka itself. Using KSQL, you can define SQL-like statements that can process one or more topics on the fly. This can give you something akin to a dynamically updating materialized database view, with the source of data being Kafka topics rather than a database. These capabilities open up some very interesting possibilities for how data is managed in distributed systems. If you'd like to explore these ideas in more detail, I can recommend *Designing Event-Driven Systems* (O'Reilly) by Ben Stopford. (I have to recommend Ben's book, as I wrote the foreword for it!) For a deeper dive into Kafka in general, I'd suggest *Kafka: The Definitive Guide* (O'Reilly) by Neha Narkhede, Gwen Shapira, and Todd Palino.

Serialization Formats

Some of the technology choices we've looked at—specifically, some of the RPC implementations—make choices for you regarding how data is serialized and deserialized. With gRPC, for example, any data sent will be converted into protocol buffer format. Many of

the technology options, though, give us a lot of freedom in terms of how we covert data for network calls. Pick Kafka as your broker of choice, and you can send messages in a variety of formats. So which format should you choose?

Textual Formats

The use of standard textual formats gives clients a lot of flexibility in how they consume resources. REST APIs most often use a textual format for the request and response bodies, even if theoretically you can quite happily send binary data over HTTP. In fact, this is how gRPC works—using HTTP underneath but sending binary protocol buffers.

JSON has usurped XML as the text serialization format of choice. You can point to a number of reasons why this has occurred, but the primary reason is that one of the main consumers of APIs is often a browser, where JSON is a great fit. JSON became popular partly as a result of the backlash against XML, and proponents cite its relative compactness and simplicity when compared to XML as another winning factor. The reality is that there's rarely a massive differential between the size of a JSON payload and that of an XML payload, especially as these payloads are typically compressed. It's also worth pointing out that some of the simplicity of JSON comes at a cost—in our rush to adopt simpler protocols, schemas went out of the window (more on that later).

Avro is an interesting serialization format. It takes JSON as an underlying structure and uses it to define a schema-based format. Avro has found a lot of popularity as a format for message payloads, partly due to its ability to send the schema as part of the payload, which can make supporting multiple different messaging formats much easier.

Personally, though, I am still a fan of XML. Some of the tool support is better. For example, if I want to extract only certain parts of the payload (a technique we'll discuss more in "Handling Change Between Microservices"), I can use XPATH, which is a well-understood standard with lots of tool support, or even CSS selectors, which many find even easier. With JSON, I have JSONPath, but this is not as widely supported. I find it odd that people pick JSON because it is nice and lightweight but then try to push concepts into it like hypermedia controls that already exist in XML. I accept, though, that I am probably in the minority here and that JSON is the format of choice for many people!

Binary Formats

While textual formats have benefits such as making it easy for humans to read them and provide a lot of interoperability with different tools and technologies, the world of binary serialization protocols is where you want to be if you start getting worried about

payload size or about the efficiencies of writing and reading the payloads. Protocol buffers have been around for a while and are often used outside the scope of gRPC—they probably represent the most popular binary serialization format for microservice-based communication.

This space is large, however, and a number of other formats have been developed with a variety of requirements in mind. Simple Binary Encoding, Cap'n Proto, and FlatBuffers all come to mind. Although benchmarks abound for each of these formats, highlighting their relevant benefits compared to protocol buffers, JSON, or other formats, benchmarks suffer from a fundamental problem in that they may not necessarily represent how you are going to use them. If you're looking to eke the last few bytes out of your serialization format, or to shave microseconds off the time taken to read or write these payloads, I strongly suggest you carry out your own comparison of these various formats. In my experience, the vast majority of systems rarely have to worry about such optimizations, as they can often achieve the improvements they are looking for by sending less data or by not making the call at all. However, if you are building an ultra-low-latency distributed system, make sure you're prepared to dive headfirst into the world of binary serialization formats.

Schemas

A discussion that comes up time and again is whether we should use schemas to define what our endpoints expose and what they accept. Schemas can come in lots of different types, and picking a serialization format will typically define which schema technology you can use. If you're working with raw XML, you'd use XML Schema Definition (XSD); if you're working with raw JSON, you'd use JSON Schema. Some of the technology choices we've touched on (specifically, a sizable subset of the RPC options) require the use of explicit schemas, so if you picked those technologies you'd have to make use of schemas. SOAP works through the use of a WSDL, while gRPC requires the use of a protocol buffer specification. Other technology choices we've explored make the use of schemas optional, and this is where things get more interesting.

As I've already discussed, I am in favor of having explicit schemas for microservice endpoints, for two key reasons. Firstly, they go a long way toward being an explicit representation of what a microservice endpoint exposes and what it can accept. This makes life easier both for developers working on the microservice and for their consumers. Schemas may not replace the need for good documentation, but they certainly can help reduce the amount of documentation required.

The other reason I like explicit schemas, though, is how they help in terms of catching accidental breakages of microservice endpoints. We'll explore how to handle changes between microservices in a moment, but it's first worth exploring the different types of breakages and the role schemas can play.

Structural Versus Semantic Contract Breakages

Broadly speaking, we can break contract breakages down into two categories—*structural* breakages and *semantic* breakages. A structural breakage is a situation in which the structure of the endpoint changes in such a way that a consumer is now incompatible—this could represent fields or methods being removed, or new required fields being added. A semantic breakage refers to a situation in which the structure of the microservices endpoint remains the same but the behavior changes in such a way as to break consumers' expectations.

Let's take a simple example. You have a highly complex `Hard Calculations` microservice that exposes a `calculate` method on its endpoint. This `calculate` method takes two integers, both of which are required fields. If you changed `Hard Calculations` such that the `calculate` method now takes only one integer, then consumers would break—they'd be sending requests with two integers that the `Hard Calculations` microservice would reject. This is an example of a structural change, and in general such changes can be easier to spot.

A semantic changes is more problematic. This is where the structure of the endpoint doesn't change but the behavior of the endpoint does. Coming back to our `calculate` method, imagine that in the first version, the two provided integers are added together and the results returned. So far, so good. Now we change `Hard Calculations` so that the `calculate` method multiplies the integers together and returns the result. The semantics of the `calculate` method have changed in a way that could break expectations of the consumers.

Should You Use Schemas?

By using schemas and comparing different versions of schemas, we can catch structural breakages. Catching semantic breakages requires the use of testing. If you don't have schemas, or if you have schemas but decide not to compare schema changes for compatibility, then the burden of catching structural breakages before you get to production also falls on testing. Arguably, the situation is somewhat analogous with static versus dynamic typing in programming languages. With a statically typed language, the types are fixed at compile time—if your code does something with an instance of a type that

isn't allowed (like calling a method that doesn't exist), then the compiler can catch that mistake. This can leave you to focus testing efforts on other sorts of problems. With a dynamically typed language, though, some of your testing will need to catch mistakes that a compiler picks up for statically typed languages.

Now, I'm pretty relaxed about static versus dynamically typed languages, and I've found myself to be very productive (relatively speaking) in both. Certainly, dynamically typed languages give you some significant benefits that for many people justify giving up on compile-time safety. Personally speaking, though, if we bring the discussion back to microservice interactions, I haven't found that a similarly balanced trade-off exists when it comes to schema versus "schemaless" communication. Put simply, I think that having an explicit schema more than offsets any perceived benefit of having schemaless communication.

Really, the question isn't actually whether you have a schema or not—it's whether or not that schema is *explicit*. If you are consuming data from a schemaless API, you still have expectations as to what data should be in there and how that data should be structured. Your code that will handle the data will be written with a set of assumptions in mind as to how that data is structured. In such a case, I'd argue that you do have a schema, but it's just totally implicit rather than explicit.[1] A lot of my desire for an explicit schema is driven by the fact that I think it's important to be as explicit as possible about what a microservice does (or doesn't) expose.

The main argument for schemaless endpoints seems to be that schemas need more work and don't give enough value. This, in my humble opinion, is partly a failure of imagination and partly a failure of good tooling to help schemas have more value when it comes to using them to catch structural breakages.

Ultimately, a lot of what schemas provide is an explicit representation of part of the structure contract between a client and a server. They help make things explicit and can greatly aid communication between teams as well as work as a safety net. In situations in which the cost of change is reduced—for example, when both client and server are owned by the same team—I am more relaxed about not having schemas.

Handling Change Between Microservices

Probably the most common question I get about microservices, after "How big should they be?" is "How do you handle versioning?" When this question gets asked, it's rarely a

query regarding what sort of numbering scheme you should use and more about how you handle changes in the contracts between microservices.

How you handle change really breaks down into two topics. In a moment, we'll look at what happens if you need to make a breaking change. But before that, let's look at what you can do to avoid making a breaking change in the first place.

Avoiding Breaking Changes

If you want to avoid making breaking changes, there are a few key ideas worth exploring, many of which we already touched on at the start of the chapter. If you can put these ideas into practice, you'll find it much easier to allow for microservices to be changed independently from one another.

Expansion changes

: Add new things to a microservice interface; don't remove old things.

Tolerant reader

: When consuming a microservice interface, be flexible in what you expect.

Right technology

: Pick technology that makes it easier to make backward-compatible changes to the interface.

Explicit interface

: Be explicit about what a microservice exposes. This makes things easier for the client and easier for the maintainers of the microservice to understand what can be changed freely.

Catch accidental breaking changes early

: Have mechanisms in place to catch interface changes that will break consumers in production before those changes are deployed.

These ideas do reinforce each other, and many build on that key concept of information hiding that we've discussed frequently. Let's look at each idea in turn.

Expansion Changes

Probably the easiest place to start is by adding only new things to a microservice contract and not removing anything else. Consider the example of adding a new field to a payload—assuming the client is in some way tolerant of such changes, this shouldn't have a material impact. Adding a new `dateOfBirth` field to a customer record should be fine, for example.

Tolerant Reader

How the consumer of a microservice is implemented can bring a lot to bear on making backward-compatible changes easy. Specifically, we want to avoid client code binding too tightly to the interface of a microservice. Let's consider an `Email` microservice whose job it is to send out emails to our customers from time to time. It gets asked to send an "order shipped" email to a customer with the ID 1234; it goes off and retrieves the customer with that ID and gets back something like the response shown in Example 5-3.

Example 5-3. Sample response from the `Customer` service

```
<customer>
  <firstname>Sam</firstname>
  <lastname>Newman</lastname>
  <email>sam@magpiebrain.com</email>
  <telephoneNumber>555-1234-5678</telephoneNumber>
</customer>
```

Now, to send the email, the `Email` microservice needs only the `firstname`, `lastname`, and `email` fields. We don't need to know the `telephoneNumber`. We want to simply pull out those fields we care about and ignore the rest. Some binding technology, especially that used by strongly typed languages, can attempt to bind *all* fields whether the consumer wants them or not. What happens if we realize that no one is using the `telephoneNumber` and we decide to remove it? This could cause consumers to break needlessly.

Likewise, what if we wanted to restructure our `Customer` object to support more details, perhaps adding some further structure, as in Example 5-4? The data our `Email` service wants is still there, with the same name, but if our code makes very explicit assumptions as to where the `firstname` and `lastname` fields will be stored, then it could break again. In this instance, we could instead use XPath to pull out the fields we care

about, allowing us to be ambivalent about where the fields are as long as we can find them. This pattern—of implementing a reader able to ignore changes we don't care about—is what Martin Fowler calls a tolerant reader.

Example 5-4. A restructured `Customer` resource: the data is all still there, but can our consumers find it?

```
<customer>
  <naming>
    <firstname>Sam</firstname>
    <lastname>Newman</lastname>
    <nickname>Magpiebrain</nickname>
    <fullname>Sam "Magpiebrain" Newman</fullname>
  </naming>
  <email>sam@magpiebrain.com</email>
</customer>
```

The example of a client trying to be as flexible as possible in consuming a service demonstrates Postel's law (otherwise known as the *robustness principle*), which states: "Be conservative in what you do, be liberal in what you accept from others." The original context for this piece of wisdom was the interaction of devices over networks, where you should expect all sorts of odd things to happen. In the context of microservice-based interactions, it leads us to try and structure our client code to be tolerant of changes to payloads.

Right Technology

As we've already explored, some technology can be more brittle when it comes to allowing us to change interfaces—I've already highlighted my own personal frustrations with Java RMI. On the other hand, some integration implementations go out of their way to make it as easy as possible for changes to be made without breaking clients. At the simple end of the spectrum, protocol buffers, the serialization format used as part of gRPC, have the concept of field number. Each entry in a protocol buffer has to define a field number, which client code expects to find. If new fields are added, the client doesn't care. Avro allows for the schema to be sent along with the payload, allowing clients to potentially interpret a payload much like a dynamic type.

At the more extreme end of the spectrum, the REST concept of HATEOAS is largely all about enabling clients to make use of REST endpoints even when they change by making

use of the previously discussed hypermedia links. This does call for you to buy into the entire HATEOAS mindset, of course.

Explicit Interface

I am a *big* fan of a microservice exposing an explicit schema that denotes what its end-points do. Having an explicit schema makes it clear to consumers what they can expect, but it also makes it much clearer to a developer working on a microservice as to what things should remain untouched to ensure you don't break consumers. Put another way, an explicit schema goes a long way toward making the boundaries of information hiding more explicit—what's exposed in the schema is by definition not hidden.

Having an explicit schema for RPC is long established and is in fact a requirement for many RPC implementations. REST, on the other hand, has typically viewed the concept of a schema as optional, to the point that I find explicit schemas for REST endpoints to be vanishingly rare. This is changing, with things like the aforementioned OpenAPI specification gaining traction, and with the JSON Schema specification also gaining in maturity.

Asynchronous messaging protocols have struggled more in this space. You can have a schema for the payload of a message easily enough, and in fact this is an area in which Avro is frequently used. However, having an explicit interface needs to go further than this. If we consider a microservice that fires events, which events does it expose? A few attempts at making explicit schemas for event-based endpoints are now underway. One is AsyncAPI, which has picked up a number of big-name users, but the one gaining most traction seems to be CloudEvents, a specification which is backed by the Cloud Native Computing Foundation (CNCF). Azure's event grid product supports the CloudEvents format, a sign of different vendors supporting this format, which should help with interoperability. This is still a fairly new space, so it will be interesting to see how things shake out over the next few years.

SEMANTIC VERSIONING

Wouldn't it be great if as a client you could look just at the version number of a service and know whether you can integrate with it? *Semantic versioning* is a specification that allows just that. With semantic versioning, each version number is in the form `MAJOR.MINOR.PATCH`. When the `MAJOR` number increments, it means that backward-incompatible changes have been made. When `MINOR` increments, new functionality has been added that should be backward compatible. Finally, a change to `PATCH` states that bug fixes have been made to existing functionality.

To see how useful semantic versioning can be, let's look at a simple use case. Our helpdesk application is built to work against version 1.2.0 of the `Customer` service. If a new feature is added, causing the `Customer` service to change to 1.3.0, our helpdesk application should see no change in behavior and shouldn't be expected to make any changes. We couldn't guarantee that we could work against version 1.1.0 of the `Customer` service, though, as we may rely on functionality added in the 1.2.0 release. We could also expect to have to make changes to our application if a new 2.0.0 release of the `Customer` service comes out.

You may decide to have a semantic version for the service, or even for an individual endpoint on a service if you are coexisting them, as detailed in the next section.

This versioning scheme allows us to pack a lot of information and expectations into just three fields. The full specification outlines in very simple terms the expectations clients can have of changes to these numbers, and it can simplify the process of communicating about whether changes should impact consumers. Unfortunately, I haven't seen this approach used enough in distributed systems to understand its effectiveness in that context—something that hasn't really changed since the first edition of this book.

Catch Accidental Breaking Changes Early

It's crucial that we pick up changes that will break consumers as soon as possible, because even if we choose the best possible technology, an innocent change of a microservice could cause consumers to break. As we've already touched on, using schemas can help us pick up structural changes, assuming we use some sort of tooling to help compare schema versions. There is a wide range of tooling out there to do this for different schema types. We have Protolock for protocol buffers, json-schema-diff-validator for JSON Schema, and openapi-diff for the OpenAPI specification.[2] More tools seem to be cropping up all the time in this space. What you're looking for, though, is something that

won't just report on the differences between two schemas but will pass or fail based on compatibility; this would allow you to fail a CI build if incompatible schemas are found, ensuring that your microservice won't get deployed.

The open source Confluent Schema Registry supports JSON Schema, Avro, and protocol buffers and is capable of comparing newly uploaded versions for backward compatibility. Although it was built to help as part of an ecosystem in which Kafka is being used, and needs Kafka to run, there is nothing to stop you from using it to store and validate schemas being used for non-Kafka-based communication.

Schema comparison tools can help us catch structural breakages, but what about semantic breakages? Or what if you aren't making use of schemas in the first place? Then we're looking at testing. This is a topic we'll explore in more detail in "Contract Tests and Consumer-Driven Contracts (CDCs)", but I wanted to highlight consumer-driven contract testing, which explicitly helps in this area—Pact being an excellent example of a tool aimed specifically at this problem. Just remember, if you don't have schemas, expect your testing to have to do more work to catch breaking changes.

If you're supporting multiple different client libraries, running tests using each library you support against the latest service is another technique that can help. Once you realize you are going to break a consumer, you have the choice to either try to avoid the break altogether or else embrace it and start having the right conversations with the people looking after the consuming services.

Managing Breaking Changes

So you've gone as far as you can to ensure that the changes you're making to a microservice's interface are backward compatible, but you've realized that you just have to make a change that will constitute a breaking change. What can you do in such a situation? You've got three main options:

Lockstep deployment

: Require that the microservice exposing the interface and all consumers of that interface are changed at the same time.

Coexist incompatible microservice versions

: Run old and new versions of the microservice side by side.

Emulate the old interface

Have your microservice expose the new interface and also emulate the old interface.

Lockstep Deployment

Of course, lockstep deployment flies in the face of independent deployability. If we want to be able to deploy a new version of our microservice with a breaking change to its interface but still do this in an independent fashion, we need to give our consumers time to upgrade to the new interface. That leads us on to the next two options I'd consider.

Coexist Incompatible Microservice Versions

Another versioning solution often cited is to have different versions of the service live at once and for older consumers to route their traffic to the older version, with newer consumers seeing the new version, as shown in Figure 5-3. This is the approach used sparingly by Netflix in situations in which the cost of changing older consumers is too high, especially in rare cases in which legacy devices are still tied to older versions of the API. I am not a fan of this idea personally, and understand why Netflix uses it rarely. First, if I need to fix an internal bug in my service, I now have to fix and deploy two different sets of services. This would probably mean I have to branch the codebase for my service, and that is always problematic. Second, it means I need smarts to handle directing consumers to the right microservice. This behavior inevitably ends up sitting in middleware somewhere, or in a bunch of `nginx` scripts, making it harder to reason about the behavior of the system. Finally, consider any persistent state our service might manage. Customers created by either version of the service need to be stored and made visible to all services, no matter which version was used to create the data in the first place. This can be an additional source of complexity.

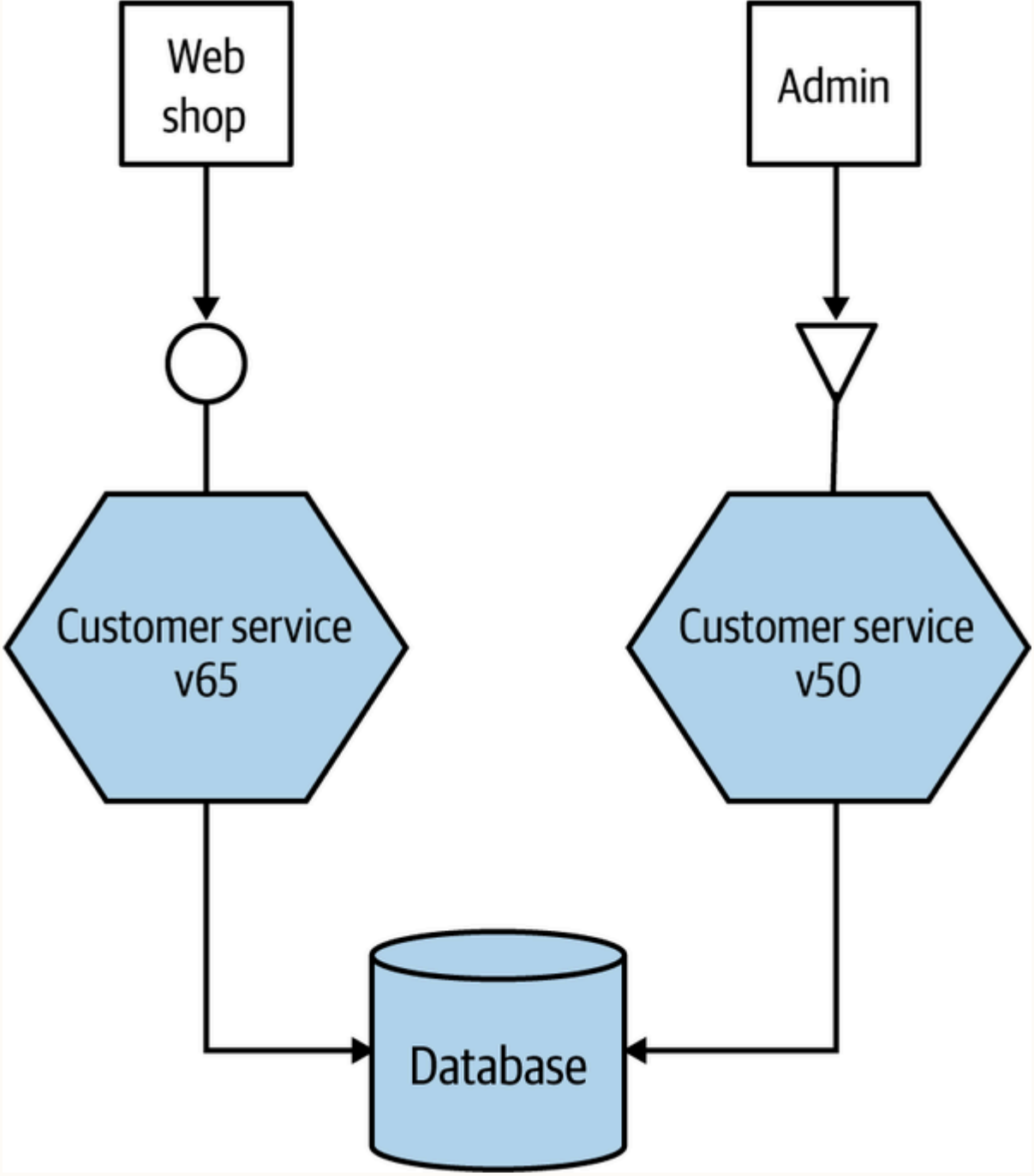


Figure 5-3. Running multiple versions of the same service to support old endpoints

Coexisting concurrent service versions for a short period of time can make perfect sense, especially when you're doing something like a canary release (we'll be discussing this pattern more in "On to Progressive Delivery"). In these situations, we may be coexisting versions for only a few minutes or perhaps hours, and we normally will have only two different versions of the service present at the same time. The longer it takes for you to get consumers upgraded to the newer version and released, the more you should look to coexist different endpoints in the same microservice rather than coexist entirely different versions. I remain unconvinced that this work is worthwhile for the average project.

Emulate the Old Interface

If we've done all we can to avoid introducing a breaking interface change, our next job is to limit the impact. The thing we want to avoid is forcing consumers to upgrade in lock-

step with us, as we always want to maintain the ability to release microservices independently of each other. One approach I have used successfully to handle this is to coexist both the old and new interfaces in the same running service. So if we want to release a breaking change, we deploy a new version of the service that exposes both the old and new versions of the endpoint.

This allows us to get the new microservice out as soon as possible, along with the new interface, while giving time for consumers to move over. Once all the consumers are no longer using the old endpoint, you can remove it along with any associated code, as shown in Figure 5-4.

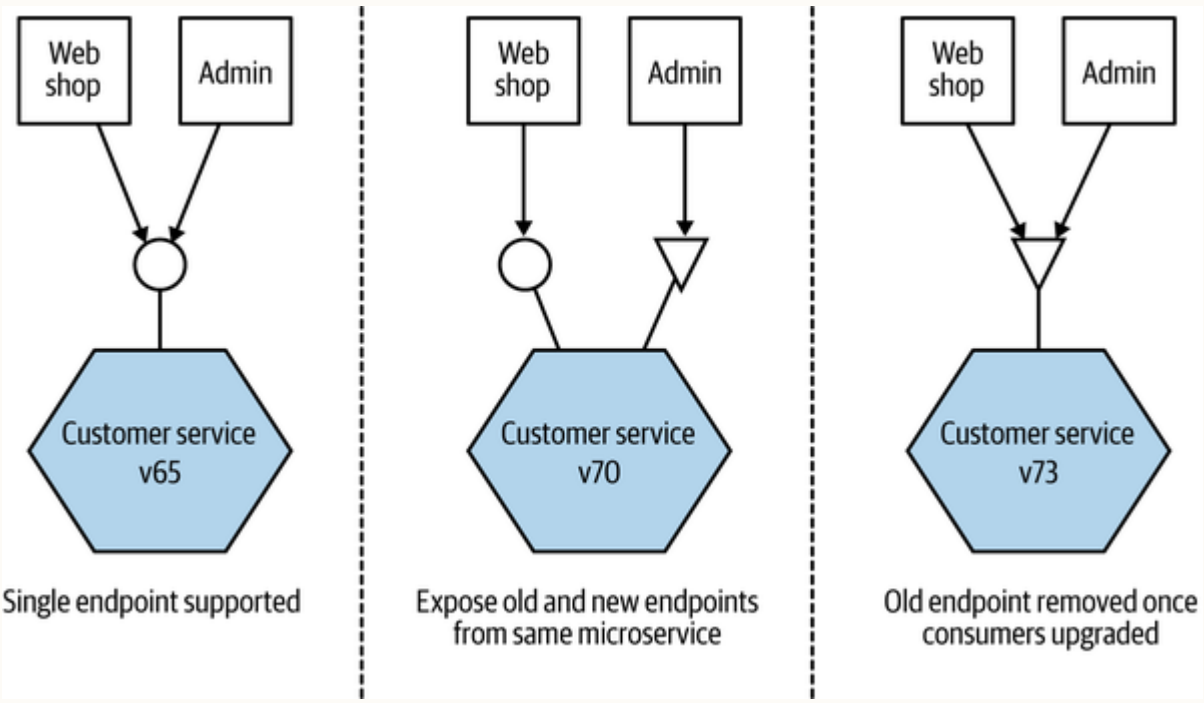


Figure 5-4. One microservice emulating the old endpoint and exposing the new backward-incompatible endpoint

When I last used this approach, we had gotten ourselves into a bit of a mess with the number of consumers we had and the number of breaking changes we had made. This meant that we were actually coexisting three different versions of the endpoint. This is not something I'd recommend! Keeping all the code around and the associated testing required to ensure they all worked was absolutely an additional burden. To make this more manageable, we internally transformed all requests to the V1 endpoint to a V2 request, and then V2 requests to the V3 endpoint. This meant we could clearly delineate what code was going to be retired when the old endpoint(s) died.

This is in effect an example of the expand and contract pattern, which allows us to phase in breaking changes. We *expand* the capabilities we offer, supporting both old and new ways of doing something. Once the old consumers do things in the new way, we *contract* our API, removing the old functionality.

If you are going to coexist endpoints, you need a way for callers to route their requests accordingly. For systems making use of HTTP, I have seen this done with both version numbers in request headers and also in the URI itself—for example, */v1/customer/* or */v2/customer/*. I'm torn as to which approach makes the most sense. On the one hand, I like URIs being opaque to discourage clients from hardcoding URI templates, but on the other hand, this approach does make things very obvious and can simplify request routing.

For RPC, things can be a little trickier. I have handled this with protocol buffers by putting my methods in different namespaces—for example, `v1.createCustomer` and `v2.createCustomer`—but when you are trying to support different versions of the same types being sent over the network, this approach can become really painful.

Which Approach Do I Prefer?

For situations in which the same team manages both the microservice and all consumers, I am somewhat relaxed about a lockstep release in limited situations. Assuming it really is a one-off situation, then doing this when the impact is limited to a single team can be justifiable. I am very cautious about this, though, as there is the danger that a one-off activity becomes business as usual, and there goes independent deployability. Use lock-step deployments too often, and you'll end up with a distributed monolith before long.

Coexisting different versions of the same microservice can be problematic, as we discussed. I'd consider doing this only in situations in which we planned to run the microservice versions side by side for only a short period of time. The reality is that when you need to give consumers time to upgrade, you could be looking at weeks or more. In other situations in which you might coexist microservice versions, perhaps as part of a blue-green deployment or canary release, the durations involved are much shorter, offsetting the downsides of this approach.

My general preference is to use emulation of old endpoints wherever possible. The challenges of implementing emulation are in my opinion much easier to deal with than those of coexisting microservice versions.

The Social Contract

Which approach you pick will be due in large part to the expectations consumers have of how these changes will be made. Keeping the old interface lying around can have a cost, and ideally you'd like to turn it off and remove associated code and infrastructure as soon as possible. On the other hand, you want to give consumers as much time as possi-

ble to make a change. And remember, in many cases the backward-incompatible changes you are making are often things that have been asked for by the consumers and/or will actually end up benefiting them. There is a balancing act, of course, between the needs of the microservice maintainers and those of the consumers, and this needs to be discussed.

I've found that in many situations, how these changes will be handled has never been discussed, leading to all sorts of challenges. As with schemas, having some degree of explicitness in how backward-incompatible changes will be made can greatly simplify things.

You don't necessarily need reams of paper and huge meetings to reach agreement on how changes will be handled. But assuming you aren't going down the route of lockstep releases, I'd suggest that both the owner and the consumer of a microservice need to be clear on a few things:

- How will you raise the issue that the interface needs to change?
- How will the consumers and microservice teams collaborate to agree on what the change will look like?
- Who is expected to do the work to update the consumers?
- When the change is agreed on, how long will consumers have to shift over to the new interface before it is removed?

Remember, one of the secrets to an effective microservice architecture is to embrace a consumer-first approach. Your microservices exist to be called by other consumers. The consumers' needs are paramount, and if you are making changes to a microservice that are going to cause upstream consumers problems, this needs to be taken into account.

In some situations, of course, it might not be possible to change the consumers. I've heard from Netflix that they had issues (at least historically) with old set-top boxes using older versions of the Netflix APIs. These set-top boxes cannot be upgraded easily, so the old endpoints have to remain available unless and until the number of older set-top boxes drops to a level at which they can have their support disabled. Decisions to stop old consumers from being able to access your endpoints can sometimes end up being financial ones—how much money does it cost you to support the old interface, balanced against how much money you make from those consumers.

Tracking Usage

Even if you do agree on a time by which consumers should stop using the old interface, would you know if they had actually stopped using it? Making sure you have logging in place for each endpoint your microservice exposes can help, as can ensuring that you have some sort of client identifier so you can chat to the team in question if you need to work with them to get them to migrate away from your old interface. This could be something as simple as asking consumers to put their identifier in the `user-agent` header when making HTTP requests, or you could require that all calls go via some sort of API gateway where clients need keys to identify themselves.

Extreme Measures

So assuming you know that a consumer is still using an old interface that you want to remove, and they are dragging their heels about moving to the new version, what can you do about it? Well, the first thing to do is talk to them. Perhaps you can lend them a hand to make the changes happen. If all else fails, and they still don't upgrade even after agreeing to do so, there are some extreme techniques I've seen used.

At one large tech company, we discussed how it handled this issue. Internally, the company had a very generous period of one year before old interfaces would be retired. I asked how it knew if consumers were still using the old interfaces, and the company replied that it didn't really bother tracking that information; after one year it just turned the old interface off. It was recognized internally that if this caused a consumer to break, it was the fault of the consuming microservice's team—they'd had a year to make the change and hadn't done so. Of course, this approach won't work for many (I said it was extreme!). It also leads to a large degree of inefficiency. By not knowing if the old interface was used, the company denied itself the opportunity to remove it before the year had passed. Personally, even if I was to suggest just turning the endpoint off after a certain period of time, I'd still definitely want tracking of who was going to be impacted.

Another extreme measure I saw was actually in the context of deprecating libraries, but it could also theoretically be used for microservice endpoints. The example given was of an old library that people were trying to retire from use inside the organization in favor of a newer, better one. Despite lots of work to move code over to use the new library, some teams were still dragging their heels. The solution was to insert a sleep in the old library so that it responded more slowly to calls (with logging to show what was happening). Over time, the team driving the deprecation just kept increasing the duration of the sleep, until eventually the other teams got the message. You obviously have to be ex-

tremely sure that you've exhausted other reasonable efforts to get consumers to upgrade before considering something like this!

DRY and the Perils of Code Reuse in a Microservice World

One of the acronyms we developers hear a lot is DRY: don't repeat yourself. Though its definition is sometimes simplified as trying to avoid duplicating code, DRY more accurately means that we want to avoid duplicating our system *behavior and knowledge*. This is very sensible advice in general. Having lots of lines of code that do the same thing makes your codebase larger than needed and therefore harder to reason about. When you want to change behavior, and that behavior is duplicated in many parts of your system, it is easy to forget everywhere you need to make a change, which can lead to bugs. So using DRY as a mantra in general makes sense.

DRY is what leads us to create code that can be reused. We pull duplicated code into abstractions that we can then call from multiple places. Perhaps we go as far as making a shared library that we can use everywhere! It turns out, though, that sharing code in a microservice environment is a bit more involved than that. As always, we have more than one option to consider.

Sharing Code via Libraries

One thing we want to avoid at all costs is overly coupling a microservice and consumers such that any small change to the microservice itself can cause unnecessary changes to the consumer. Sometimes, however, the use of shared code can create this very coupling. For example, at one client we had a library of common domain objects that represented the core entities in use in our system. This library was used by all the services we had. But when a change was made to one of them, all services had to be updated. Our system communicated via message queues, which also had to be drained of their now *invalid* contents, and woe betide you if you forgot.

If your use of shared code ever leaks outside your service boundary, you have introduced a potential form of coupling. Using common code like logging libraries is fine, as they are internal concepts that are invisible to the outside world. The website realestate.com.au makes use of a tailored service template to help bootstrap new service creation. Rather than make this code shared, the company copies it for every new service to ensure that coupling doesn't leak in.

The really important point about sharing code via libraries is that you cannot update all uses of the library at once. Although multiple microservices might all use the same library, they do so typically by packaging that library into the microservice deployment. To upgrade the version of the library being used, you'd therefore need to redeploy the microservice. If you want to update the same library everywhere at exactly the same time, it could lead to a widespread deployment of multiple different microservices all at the same time, with all the associated headaches.

So if you are using libraries for code reuse across microservice boundaries, you have to accept that multiple different versions of the same library might be out there at the same time. You can of course look to update all of these to the last version over time, but as long as you are OK with this fact, then by all means reuse code via libraries. If you really do need to update that code for all users at exactly the same time, then you'll actually want to look at reusing code via a dedicated microservice instead.

There is one specific use case associated with reuse through libraries that is worth exploring further, though.

Client libraries

I've spoken to more than one team that has insisted that creating client libraries for your services is an essential part of creating services in the first place. The argument is that this makes it easy to use your service and avoids the duplication of code required to consume the service itself.

The problem, of course, is that if the same people create both the server API and the client API, there is a danger that logic that should exist on the server will start leaking into the client. I should know: I've done this myself. The more logic that creeps into the client library, the more cohesion starts to break down, and you find yourself having to change multiple clients to roll out fixes to your server. You also limit technology choices, especially if you mandate that the client library has to be used.

A model I like for client libraries is the one for Amazon Web Services (AWS). The underlying SOAP or REST web service calls can be made directly, but everyone ends up using just one of the various existing software development kits (SDKs), which provide abstractions over the underlying API. These SDKs, though, are written by the wider community, or else by people inside AWS other than those who work on the API itself. This degree of separation seems to work and avoids some of the pitfalls of client libraries. Part of the reason this works so well is that the client is in charge of when the upgrade happens. If you go down the path of client libraries yourself, make sure this is the case.

Netflix in particular places special emphasis on the client library, but I worry that people view that purely through the lens of avoiding code duplication. In fact, the client libraries used by Netflix are as much if not more about ensuring reliability and scalability of their systems. The Netflix client libraries handle service discovery, failure modes, logging, and other aspects that aren't actually about the nature of the service itself. Without these shared clients, it would be hard to ensure that each piece of client/server communications behaved well at the massive scale at which Netflix operates. Their use at Netflix has certainly made it easy to get up and running and increase productivity while also ensuring the system behaves well. However, according to at least one person at Netflix, over time this has led to a degree of coupling between client and server that has been problematic.

If the client library approach is something you're thinking about, it can be important to separate out client code to handle the underlying transport protocol, which can deal with things like service discovery and failure, from things related to the destination service itself. Decide whether or not you are going to insist on the client library being used, or if you'll allow people using different technology stacks to make calls to the underlying API. And finally, make sure that the clients are in charge of when to upgrade their client libraries: we need to ensure we maintain the ability to release our services independently of each other!

Service Discovery

Once you have more than a few microservices lying around, your attention inevitably turns to knowing where on earth everything is. Perhaps you want to know what is running in a given environment so you know what you should be monitoring. Maybe it's as simple as knowing where your `Accounts` microservice is so that its consumers know where find it. Or perhaps you just want to make it easy for developers in your organization to know what APIs are available so they don't reinvent the wheel. Broadly speaking, all of these use cases fall under the banner of *service discovery*. And as always with microservices, we have quite a few different options at our disposal for dealing with it.

All of the solutions we'll look at handle things in two parts. First, they provide some mechanism for an instance to register itself and say, "I'm here!" Second, they provide a way to find the service once it's registered. Service discovery gets more complicated, though, when we are considering an environment in which we are constantly destroying and deploying new instances of services. Ideally, we'd want whatever solution we pick to cope with this.

Let's look at some of the most common solutions to service delivery and consider our options.

Domain Name System (DNS)

It's nice to start simple. DNS lets us associate a name with the IP address of one or more machines. We could decide, for example, that our `Accounts` microservice is always found at *accounts.musiccorp.net*. We would then have that entry point to the IP address of the host running that microservice, or perhaps we'd have it resolve to a load balancer that is distributing load across a number of instances. This means we'd have to handle updating these entries as part of deploying our service.

When dealing with instances of a service in different environments, I have seen a convention-based domain template work well. For example, we might have a template defined as *<servicename>-<environment>.musiccorp.net*, giving us entries like *accounts-uat.musiccorp.net* or *accounts-dev.musiccorp.net*.

A more advanced way of handling different environments is to have different domain name servers for these environments. So I could assume that *accounts.musiccorp.net* is where I always find the `Accounts` microservice, but it could resolve to different hosts depending on where I do the lookup. If you already have your environments sitting in different network segments and are comfortable with managing your own DNS servers and entries, this could be quite a neat solution, but it is a lot of work if you aren't getting other benefits from this setup.

DNS has a host of advantages, the main one being it is such a well-understood and well-used standard that almost any technology stack will support it. Unfortunately, while a number of services exist for managing DNS inside an organization, few of them seem designed for an environment in which we are dealing with highly disposable hosts, making updating DNS entries somewhat painful. Amazon's Route 53 service does a pretty good job of this, but I haven't seen a self-hosted option that is as good yet, although (as we'll discuss shortly) some dedicated service discovery tools like Consul may help us here. Aside from the problems in updating DNS entries, the DNS specification itself can cause us some issues.

DNS entries for domain names have a *time to live* (TTL). This is how long a client can consider the entry fresh. When we want to change the host to which the domain name refers, we update that entry, but we have to assume that clients will be holding on to the old IP for *at least* as long as the TTL states. DNS entries can get cached in multiple

places (even the JVM will cache DNS entries unless you tell it not to), and the more places they are cached in, the more stale the entry can be.

One way to work around this problem is to have the domain name entry for your service point to a load balancer, which in turn points to the instances of your service, as shown in Figure 5-5. When you deploy a new instance, you can take the old one out of the load-balancer entry and add the new one. Some people use DNS round-robining, where the DNS entries themselves refer to a group of machines. This technique is extremely problematic, as the client is hidden from the underlying host and therefore cannot easily stop routing traffic to one of the hosts should it become sick.

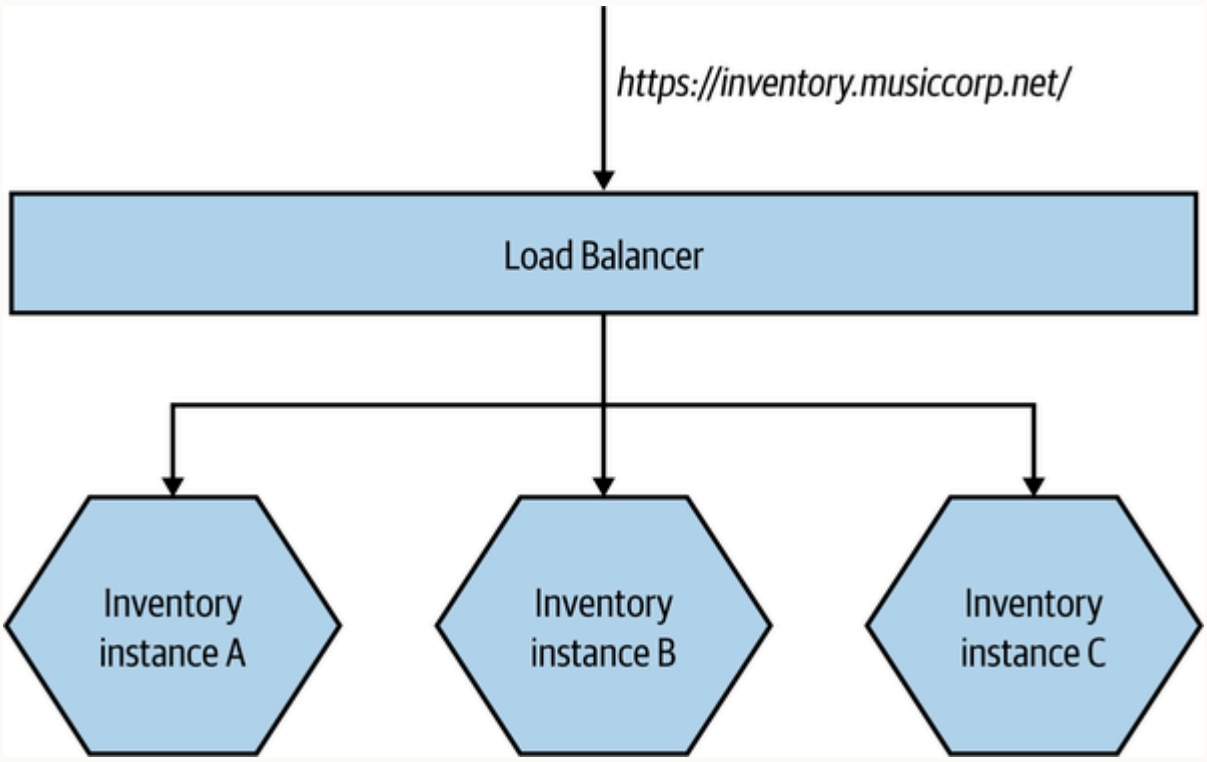


Figure 5-5. Using DNS to resolve to a load balancer to avoid stale DNS entries

As mentioned, DNS is well understood and widely supported. But it does have one or two downsides. I would suggest you investigate whether it is a good fit for you before picking something more complex. For a situation in which you have only single nodes, having DNS refer directly to hosts is probably fine. But for those situations in which you need more than one instance of a host, have DNS entries resolve to load balancers that can handle putting individual hosts into and out of service as appropriate.

Dynamic Service Registries

The downsides of DNS as a way of finding nodes in a highly dynamic environment have led to a number of alternative systems, most of which involve the service registering itself with some central registry, which in turn offers the ability to look up these services later on. Often, these systems do more than just providing service registration and dis-

covery, which may or may not be a good thing. This is a crowded field, so we'll just look at a few options to give you a sense of what is available.

ZooKeeper

ZooKeeper was originally developed as part of the Hadoop project. It is used for an almost bewildering array of use cases, including configuration management, synchronizing data between services, leader election, message queues, and (usefully for us) as a naming service.

Like many similar types of systems, ZooKeeper relies on running a number of nodes in a cluster to provide various guarantees. This means you should expect to be running at least three Zookeeper nodes. Most of the smarts in ZooKeeper are around ensuring that data is replicated safely between these nodes, and that things remain consistent when nodes fail.

At its heart, ZooKeeper provides a hierarchical namespace for storing information. Clients can insert new nodes in this hierarchy, change them, or query them. Furthermore, they can add watches to nodes to be told when they change. This means we could store the information about where our services are located in this structure and as a client be told when they change. ZooKeeper is often used as a general configuration store, so you could also store service-specific configuration in it, allowing you to do tasks like dynamically changing log levels or turning off features of a running system.

In reality, better solutions exist for dynamic service registration, to the extent that I would actively avoid ZooKeeper for this use case nowadays.

Consul

Like ZooKeeper, Consul supports both configuration management and service discovery. But it goes further than ZooKeeper in providing more support for these key use cases. For example, it exposes an HTTP interface for service discovery, and one of Consul's killer features is that it actually provides a DNS server out of the box; specifically, it can serve SRV records, which give you both an IP and a port for a given name. This means if part of your system uses DNS already and can support SRV records, you can just drop in Consul and start using it without any changes to your existing system.

Consul also builds in other capabilities that you might find useful, such as the ability to perform health checks on nodes. Thus Consul could well overlap the capabilities provided by other dedicated monitoring tools, although you would more likely use Consul as

a source of this information and then pull it into a more comprehensive monitoring setup.

Consul uses a RESTful HTTP interface for everything from registering a service to querying the key/value store or inserting health checks. This makes integration with different technology stacks very straightforward. Consul also has a suite of tools that work well with it, further improving its usefulness. One example is consul-template, which provides a way to update text files based on entries in Consul. At first glance, this doesn't seem that interesting, until you consider the fact that with consul-template you can now change a value in Consul—perhaps the location of a microservice, or a configuration value—and have configuration files all across your system dynamically update. Suddenly, any program that reads its configuration from a text file can have its text files updated dynamically without needing to know anything about Consul itself. A great use case for this would be dynamically adding or removing nodes to a load balancer pool using a software load balancer like HAProxy.

Another tool that integrates well with Consul is Vault, a secrets management tool we'll revisit in "Secrets". Secrets management can be painful, but the combination of Consul and Vault can certainly make life easier.

etcd and Kubernetes

If you're running on a platform that manages container workloads for you, chances are you already have a service discovery mechanism provided for you. Kubernetes is no different, and it comes partly from etcd, a configuration management store bundled with Kubernetes. etcd has capabilities similar to those of Consul, and Kubernetes uses it for managing a wide array of configuration information.

We'll explore Kubernetes in more detail in "Kubernetes and Container Orchestration", but in a nutshell, the way service discovery works on Kubernetes is that you deploy a container in a pod, and then a service dynamically identifies which pods should be part of a service by pattern matching on metadata associated with the pod. It's a pretty elegant mechanism and can be very powerful. Requests to a service will then get routed to one of the pods that make up that service.

The capabilities that you get out of the box with Kubernetes may well result in you just wanting to make do with what comes with the core platform, eschewing the use of dedicated tools like Consul, and for many this makes a lot of sense, especially if the wider ecosystem of tooling around Consul isn't of interest to you. However, if you're running in a mixed environment, where you have workloads running on Kubernetes and elsewhere,

then having a dedicated service discovery tool that could be used across both platforms may be the way to go.

Rolling your own

One approach I have used myself and seen used elsewhere is to roll your own system. On one project, we were making heavy use of AWS, which offers the ability to add tags to instances. When launching service instances, I would apply tags to help define what the instance was and what it was used for. These allowed for some rich metadata to be associated with a given host—for example:

- service = accounts
- environment = production
- version = 154

I then used the AWS APIs to query all the instances associated with a given AWS account so I could find the machines I cared about. Here, AWS itself is handling the storing of the metadata associated with each instance and providing us with the ability to query it. I then built command-line tools for interacting with these instances and provided graphical interfaces to view instance status at a glance. All of this becomes quite a simple undertaking if you can programmatically gather information about the service interfaces.

The last time I did this we didn't go as far as having services use the AWS APIs to find their service dependencies, but there is no reason why you couldn't. Obviously, if you want upstream services to be alerted when the location of a downstream service changes, you're on your own.

Nowadays, this is not the route I would go. The crop of tooling in this space is mature enough that this would be a case of not just reinventing the wheel but recreating a much worse wheel.

Don't Forget the Humans!

The systems we've looked at so far make it easy for a service instance to register itself and look up other services it needs to talk to. But as humans we sometimes want this information too. Making the information available in ways that allow for humans to consume it, perhaps by using APIs to pull this detail into humane registries (a topic we'll look at in a moment), can be vital.

Service Meshes and API Gateways

Few areas of technology associated with microservices have had as much attention, hype, and confusion around them as that of service meshes and API gateways. Both have their place, but confusingly, they can also overlap in responsibilities. The API gateway in particular is prone to misuse (and mis-selling), so it's important that we understand how these types of technology can fit into our microservice architecture. Rather than attempt to deliver a detailed view as to what you can do with these products, I instead want to provide an overview of where they fit in, how they can help, and some pitfalls to avoid.

In typical data center speak, we'd talk about "east-west" traffic as being inside a data center, with "north-south" traffic relating to interactions that enter or leave the data center from the outside world. From the point of view of networking, what a data center is has become a somewhat blurry concept, so for our purposes we'll talk more broadly about a networked perimeter. This could relate to an entire data center, a Kubernetes cluster, or perhaps just a virtual networking concept like a group of machines running on the same virtual LAN.

Speaking generally, an API gateway sits on the perimeter of your system and deals with north-south traffic. Its primary concerns are managing access from the outside world to your internal microservices. A service mesh, on the other hand, deals very narrowly with communication between microservices inside your perimeter—the east-west traffic—as Figure 5-6 shows.

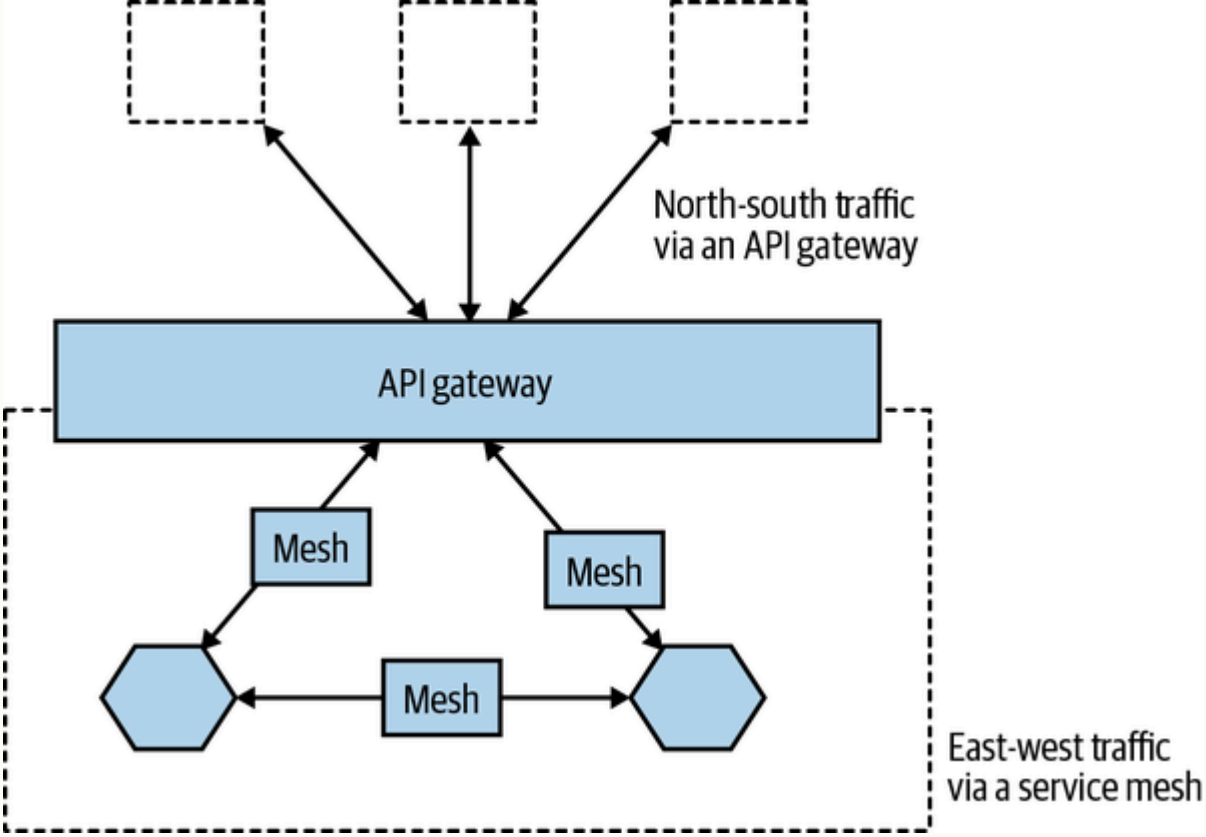


Figure 5-6. An overview of where API gateways and service meshes are used

Service meshes and API gateways can potentially allow microservices to share code without requiring the creation of new client libraries or new microservices. Put (very) simply, service meshes and API gateways can work as proxies between microservices. This can mean that they may be used to implement some microservice-agnostic behavior that might otherwise have to be done in code, such as service discovery or logging.

If you are using an API gateway or a service mesh to implement shared, common behavior for your microservices, it's essential that this behavior is totally generic—in other words, that the behavior in the proxy bears no relation to any specific behavior of an individual microservice.

Now, after explaining that, I also have to explain that the world isn't always as clear-cut as that. A number of API gateways attempt to provide capabilities for east-west traffic too, but that's something we'll discuss shortly. Firstly, let's look at API gateways and the sorts of things they can do.

API Gateways

Being focused more on north-south traffic, the API gateway's main concern in a microservices environment is mapping requests from external parties to internal microservices. This responsibility is akin to what you could achieve with a simple HTTP proxy, and in fact API gateways typically build more features on top of existing HTTP proxy products, and they largely function as reverse proxies. In addition, API gateways can be used

to implement mechanisms like API keys for external parties, logging, rate limiting, and the like. Some API gateway products will also provide developer portals, often targeted at external consumers.

Part of the confusion around the API gateway has to do with history. A while back, there was a huge amount of interest in what was called "the API economy." The industry had started to understand the power of offering APIs to managed solutions, from SaaS products like Salesforce to platforms like AWS, as it was clear that an API gave customers a lot more flexibility in how their software was used. This caused a number of people to start looking at the software they already had and consider the benefits of exposing that functionality to their customers not just via a GUI but also via an API. The hope was that this would open up larger market opportunities, and, well, make more money. Amid this interest, a crop of API gateway products emerged to help make achieving those goals possible. Their featureset leaned heavily on managing API keys for third parties, enforcing rate limits, and tracking use for chargeback purposes. The reality is that while APIs have absolutely been shown to be an excellent way to deliver services to some customers, the size of the API economy wasn't quite as big as some had hoped, and a lot of companies found that they had purchased API gateway products laden with features they never actually needed.

Much of the time, all an API gateway is actually being used for is to manage access to an organization's microservices from its own GUI clients (web pages, native mobile applications) via the public internet. There is no "third party" in the mix here. The need for some form of an API gateway for Kubernetes is essential, as Kubernetes natively handles networking only within the cluster and does nothing about handling communication to and from the cluster itself. But in such a use case, an API gateway designed for external third-party access is huge overkill.

So if you want an API gateway, be really clear in what you expect from it. In fact, I'd go a bit further and say that you should probably avoid having an API gateway that does too much. But we'll get to that next.

Where to use them

Once you start to understand what sort of use cases you have, it becomes a bit easier to see what type of gateway you need. If it's just a case of exposing microservices running in Kubernetes, you could run your own reverse proxies—or better yet, you could look at a focused product like Ambassador, which was built from the ground up with that use case in mind. If you really find yourself needing to manage large numbers of third-party users accessing your API, then there are likely other products to look at. It's possible, in fact,

that you may end up with more than one gateway in the mix to better handle separation of concerns, and I can see that being sensible in many situations, although the usual caveats around increasing the overall system complexity and increasing network hops still apply.

I have been involved from time to time in working directly with vendors to help with tool selection. I can say without any hesitation that I have experienced more mis-selling and poor or cutthroat behavior in the API gateway space than in any other—and as a result you won't find references to some vendor products in this chapter. I've put a lot of that down to VC-backed companies that had built a product for the boom times of the API economy, only to find that market doesn't exist, and so they are fighting on two fronts: they are fighting over the small number of users that actually need what the more complex gateways are offering, while also losing business to more focused API gateway products that are built for the vast majority of simpler needs.

What to avoid

Partly due to the apparent desperation of some API gateway vendors, all sorts of claims have been made for what these products can do. This has led to a lot of misuse of these products, and in turn to an unfortunate distrust of what is fundamentally quite a simple concept. Two key examples I've seen of misuse of API gateways is for call aggregation and protocol rewriting, but I've also seen a wider push to use API gateways for in-perimeter (east-west) calls too.

In this chapter we've already briefly looked at the usefulness of a protocol like GraphQL to help us in a situation in which we need to make a number of calls and then aggregate and filter the results, but people are often tempted to solve this problem in API gateway layers too. It starts off innocently enough: you combine a couple of calls and return a single payload. Then you start making another downstream call as part of the same aggregated flow. Then you start wanting to add conditional logic, and before long you realize that you've baked core business processes into a third-party tool that is ill suited to the task.

If you find yourself needing to do call aggregation and filtering, then look at the potential of GraphQL or the BFF pattern, which we'll cover in Chapter 14. If the call aggregation you are performing is fundamentally a business process, then this is better done through an explicitly modeled saga, which we'll cover in Chapter 6.

Aside from the aggregation angle, protocol rewriting is also often pushed as something API gateways should be used for. I remember one unnamed vendor very aggressively

promoting the idea that its product could "change any SOAP API into a REST API." Firstly, REST is an entire architectural mindset that cannot simply be implemented in a proxy layer. Secondly, protocol rewriting, which is fundamentally what this is trying to do, shouldn't be done in intermediate layers, as it's pushing too much behavior to the wrong place.

The main problem with both the protocol rewriting capability and the implementation of call aggregation inside API gateways is that we are violating the rule of keeping the pipes dumb, and the endpoints smart. The "smarts" in our system want to live in our code, where we can have full control over them. The API gateway in this example is a pipe—we want it as simple as possible. With microservices, we are pushing for a model in which changes can be made and more easily released through independent deployability. Keeping smarts in our microservices helps this. If we now also have to make changes in intermediate layers, things become more problematic. Given the criticality of API gateways, changes to them are often tightly controlled. It seems unlikely that individual teams will be given free rein to self-service change these often centrally managed services. What does that mean? Tickets. To roll out a change to your software, you end up having the API gateway team make changes for you. The more behavior you leak into API gateways (or into enterprise service buses), the more you run the risk of handoffs, increased coordination, and slowed delivery.

The last issue is the use of an API gateway as an intermediary for all inter-microservice calls. This can be extremely problematic. If we insert an API gateway or a normal network proxy between two microservices, then we have normally added at least a single network hop. A call from microservice A to microservice B first goes from A to the API gateway and then from the API gateway to B. We have to consider the latency impact of the additional network call and the overhead of whatever the proxy is doing. Service meshes, which we explore next, are much better placed to solve this problem.

Service Meshes

With a service mesh, common functionality associated with inter-microservice communication is pushed into the mesh. This reduces the functionality that a microservice needs to implement internally, while also providing consistency across how certain things are done.

Common features implemented by service meshes include mutual TLS, correlation IDs, service discovery and load balancing, and more. Often this type of functionality is fairly generic from one microservice to the next, so we'd end up making use of a shared library to handle it. But then you have to deal with what happens if different microservices have

different versions of the libraries running, or what happens if you have microservices written in different runtimes.

Historically at least, Netflix would mandate that all nonlocal network communication had to be done JVM to JVM. This was to ensure that the tried and tested common libraries that are a vital part of managing effective communication between microservices could be reused. With the use of a service mesh, though, we have the possibility of reusing common inter-microservice functionality across microservices written in different programming languages. Service meshes can also be incredibly useful in implementing standard behavior across microservices created by different teams—and the use of a service mesh, especially on Kubernetes, has increasingly become an assumed part of any given platform you might create for self-service deployment and management of microservices.

Making it easy to implement common behavior across microservices is one of the big benefits of a service mesh. If this common functionality was implemented solely through shared libraries, changing this behavior would require every microservice to pull in a new version of said libraries and be deployed before that change is live. With a service mesh, you have much more flexibility in rolling out changes in terms of inter-microservice communication without requiring a rebuild and redeploy.

How they work

In general, we'd expect to have less north-south traffic than east-west traffic with a microservice architecture. A single north-south call—placing an order, for example—could result in multiple east-west calls. This means that when considering any sort of proxy for in-perimeter calls, we have to be aware of the overhead these additional calls can cause, and this is a core consideration in terms of how service meshes are built.

Service meshes come in different shapes and sizes, but what unites them is that their architecture is based on trying to limit the impact caused by calls to and from the proxy. This is achieved primarily by distributing the proxy processes to run on the same physical machines as the microservice instances, to ensure that the number of remote network calls is limited. In Figure 5-7 we see this in action—the `Order Processor` is sending a request to the `Payment` microservice. This call is first routed locally to a proxy instance running on the same machine as `Order Processor`, before continuing to the `Payment` microservice via its local proxy instance. The `Order Processor` thinks it's making a normal network call, unaware that the call is routed locally on the machine, which is significantly faster (and also less prone to partitions).

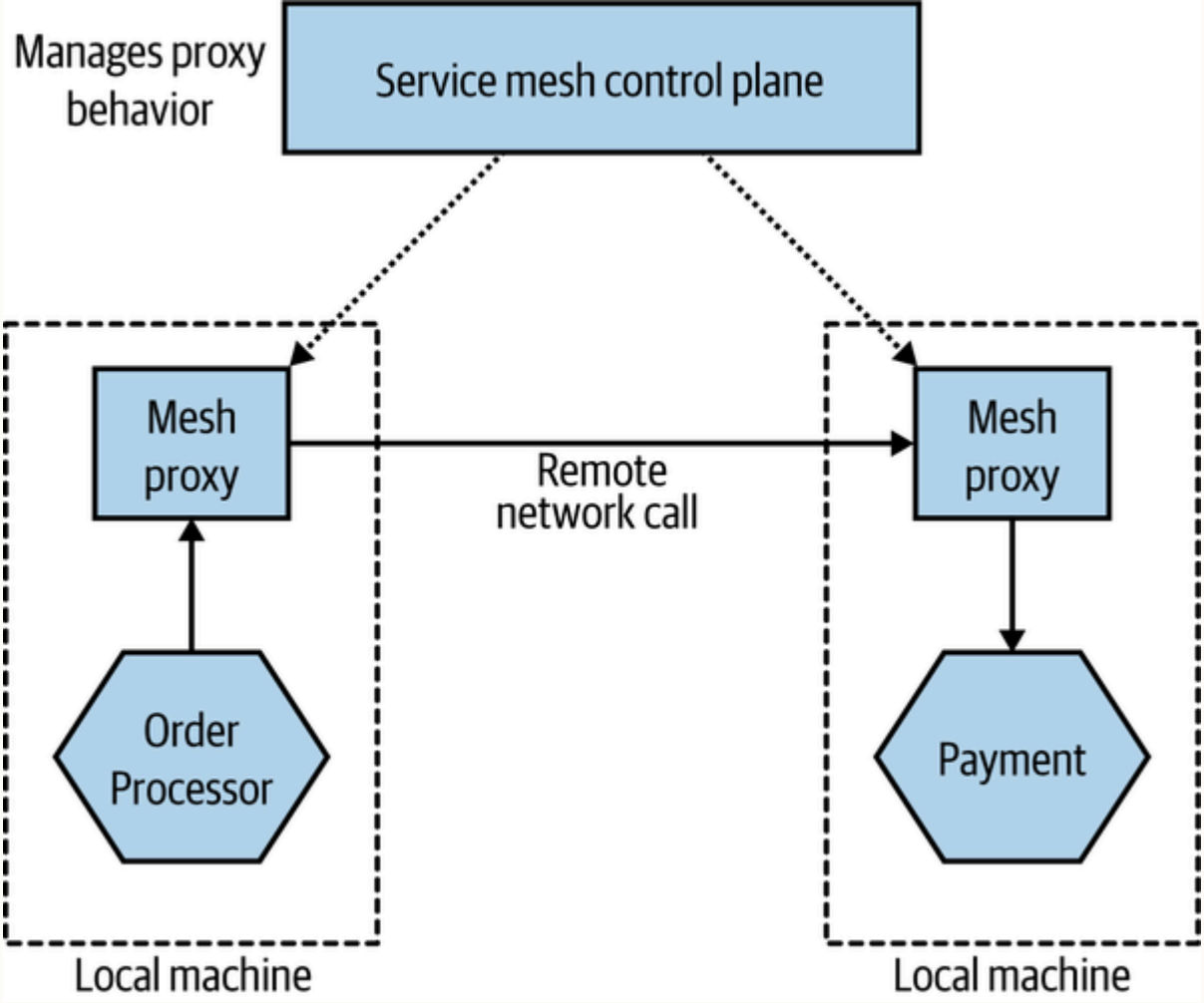


Figure 5-7. A service mesh is deployed to handle all direct inter-microservice communication

A control plane would sit on top of the local mesh proxies, acting as both a place in which the behavior of these proxies can be changed and a place in which you can collect information about what the proxies are doing.

When deploying on Kubernetes, you would deploy each microservice instance in a pod with its own local proxy. A single pod is always deployed as a single unit, so you always know that you have a proxy available. Moreover, a single proxy dying would impact only that one pod. This setup also allows you to configure each proxy differently for different purposes. We'll look at these concepts in more detail in "Kubernetes and Container Orchestration".

Many service mesh implementations use the Envoy proxy for the basis of these locally running processes. Envoy is a lightweight C++ proxy often used as the building block for service meshes and other types of proxy-based software—it is an important building block for Istio and Ambassador, for example.

These proxies are in turn managed by a control plane. This will be a set of software that helps you see what is going on and control what is being done. When using a service mesh to implement mutual TLS, for example, the control plane would be used to distribute client and server certificates.

Aren't service meshes smart pipes?

So all this talk of pushing common behavior into a service mesh might have alarm bells ringing for some of you. Isn't this approach open to the same sorts of problems as enterprise service buses or overly bloated API gateways? Aren't we at risk of pushing too many "smarts" into our service mesh?

The key thing to remember here is that the common behavior we are putting into the mesh is not specific to any one microservice. No business functionality has leaked to the outside. We're configuring generic things like how request time-outs are handled. In terms of common behavior that might want to be tweaked on a per-microservice basis, that's typically something that is well catered for, without the need for work to be done on a central platform. For example, with Istio, I can define my time-out requirements on a self-service basis just by changing my service definition.

Do you need one?

When the use of service meshes first started becoming popular, just after the release of the first edition of this book, I saw a lot of merit in the idea but also saw a lot of churn in the space. Different deployment models were suggested, built, and then dropped, and the number of companies offering solutions in this space increased drastically; but even for those tools that had been around for a long time, there was an apparent lack of stability. Linkerd, which arguably did as much as anyone to pioneer this space, totally rebuilt its product from scratch in the shift from v1 to v2. Istio, which was the Google-anointed service mesh, took years to get to an initial 1.0 release, and even still it had significant subsequent changes in its architecture (moving somewhat ironically, although sensibly, to a more monolithic deployment model for its control plane).

For much of the last five years, when I have been asked "Should we get a service mesh?" my advice has been "If you can afford to wait six months before making a choice, then wait six months." I was sold on the idea but concerned about the stability. And something like a service mesh is not where I personally want to take a lot of risks—it's so key, so essential to everything working well. You're putting it on your critical path. It's up there with selecting a message broker or cloud provider in terms of how seriously I'd take it.

Since then, I'm happy to say, this space has matured. The churn to some extent has slowed, but we still have a (healthy) plurality of vendors. That said, service meshes aren't for everyone. Firstly, if you aren't on Kubernetes, your options are limited. Secondly, they do add complexity. If you have five microservices, I don't think you can

easily justify a service mesh (it's arguable as to whether you can justify Kubernetes if you only have five microservices!). For organizations that have more microservices, especially if they want the option for those microservices to be written in different programming languages, service meshes are well worth a look. Do your homework, though—switching between service meshes is painful!

Monzo is one organization that has spoken openly about how its use of a service mesh was essential in allowing it to run its architecture at the scale it does. Its use of version 1 of Linkerd to help manage inter-microservice RPC calls proved hugely beneficial. Interestingly, Monzo had to handle the pain of a service mesh migration to help it achieve the scale it needed when the older architecture of Linkerd v1 no longer met its requirements. In the end it moved effectively to an in-house service mesh making use of the Envoy proxy.

What About Other Protocols?

API gateways and service meshes are primarily used to handle HTTP-related calls. So REST, SOAP, gRPC, and the like can be managed via these products. Things get a bit more murky, though, when you start looking at communication via other protocols, like the use of message brokers such as Kafka. Typically, at this point the service mesh gets bypassed—communication is done directly with the broker itself. This means that you cannot assume your service mesh is able to work as an intermediary for all calls between microservices.

Documenting Services

By decomposing our systems into finer-grained microservices, we're hoping to expose lots of seams in the form of APIs that people can use to do many hopefully wonderful things. If you get our discovery right, we know where things are. But how do we know what those things do or how to use them? One option is obviously to have documentation about the APIs. Of course, documentation can often be out of date. Ideally, we'd ensure that our documentation is always up to date with the microservice API and make it easy to see this documentation when we know where a service endpoint is.

Explicit Schemas

Having explicit schemas does go a long way toward making it easier to understand what any given endpoint exposes, but by themselves they are often not enough. As we've already discussed, schemas help show the structure, but they don't go very far in helping

communicate the behavior of an endpoint, so good documentation could still be required to help consumers understand how to use an endpoint. It's worth noting, of course, that if you decide not to use an explicit schema, your documentation will end up doing *more* work. You'll need to explain what the endpoint does and also document the structure and detail of the interface. Moreover, without an explicit schema, detecting whether your documentation is up to date with the real endpoints is more difficult. Stale documentation is an ongoing problem, but at least an explicit schema gives you more chance of it being up to date.

I've already introduced OpenAPI as a schema format, but it also is very effective in providing documentation, and a lot of open source and commercial tools now exist that can support consuming the OpenAPI descriptors to help create useful portals to allow developers to read the documentation. It's worth noting that the open source portals for viewing OpenAPI seem somewhat basic—I struggled to find one that supported search functionality, for example. For those on Kubernetes, Ambassador's developer portal is especially interesting. Ambassador is already a popular choice as an API gateway for Kubernetes, and its Developer Portal has the ability to autodiscover available OpenAPI endpoints. The idea of deploying a new microservice and having its documentation automatically available greatly appeals to me.

In the past we've lacked good support for documenting event-based interfaces. Now at least we have options. The AsyncAPI format started off as an adaptation of OpenAPI, and we also now have CloudEvents, which is a CNCF project. I've not used either in anger (that is, in a real setting), but I'm more drawn to CloudEvents purely because it seems to have a wealth of integration and support, due in large part to its association with the CNCF. Historically, at least, CloudEvents seemed to be more restrictive in terms of the event format compared to AsyncAPI, with only JSON being properly supported, until protocol buffer support was recently reintroduced after previously being removed; so that may be a consideration.

The Self-Describing System

During the early evolution of SOA, standards like Universal Description, Discovery, and Integration (UDDI) emerged to help us make sense of what services were running. These approaches were fairly heavyweight, which led to alternative techniques to try to make sense of our systems. Martin Fowler has discussed the concept of the humane registry, a much more lightweight approach in which humans can record information about the services in the organization in something as basic as a wiki.

Getting a picture of our system and how it is behaving is important, especially when we're at scale. We've covered a number of different techniques that will help us gain understanding directly from our system. By tracking the health of our downstream services together with correlation IDs to help us see call chains, we can get real data in terms of how our services interrelate. Using service discovery systems like Consul, we can see where our microservices are running. Mechanisms like OpenAPI and CloudEvents can help us see what capabilities are being hosted on any given endpoint, while our health check pages and monitoring systems let us know the health of both the overall system and individual services.

All of this information is available programmatically. All of this data allows us to make our humane registry more powerful than a simple wiki page that will no doubt get out of date. Instead, we should use it to harness and display all the information our system will be emitting. By creating custom dashboards, we can pull together the vast array of information that is available to help us make sense of our ecosystem.

By all means, start with something as simple as a static web page or wiki that perhaps scrapes in a bit of data from the live system. But look to pull in more and more information over time. Making this information readily available is a key tool for managing the emerging complexity that will come from running these systems at scale.

I've spoken with a number of companies that have had these issues and that ended up creating simple in-house registries to help collate metadata around services. Some of these registries simply crawl source code repositories, looking for metadata files to build up a list of services out there. This information can be merged with real data coming from service discovery systems like Consul or etcd to build up a richer picture of what is running and whom you could speak to about it.

The *Financial Times* created Biz Ops to help address this problem. The company has several hundred services developed by teams all over the world. The Biz Ops tool (Figure 5-8) gives the company a single place where you can find out lots of useful information about its microservices, in addition to information about other IT infrastructure services such as networks and file servers. Built on top of a graph database, Biz Ops has a lot of flexibility about what data it gathers and how the information can be modeled.

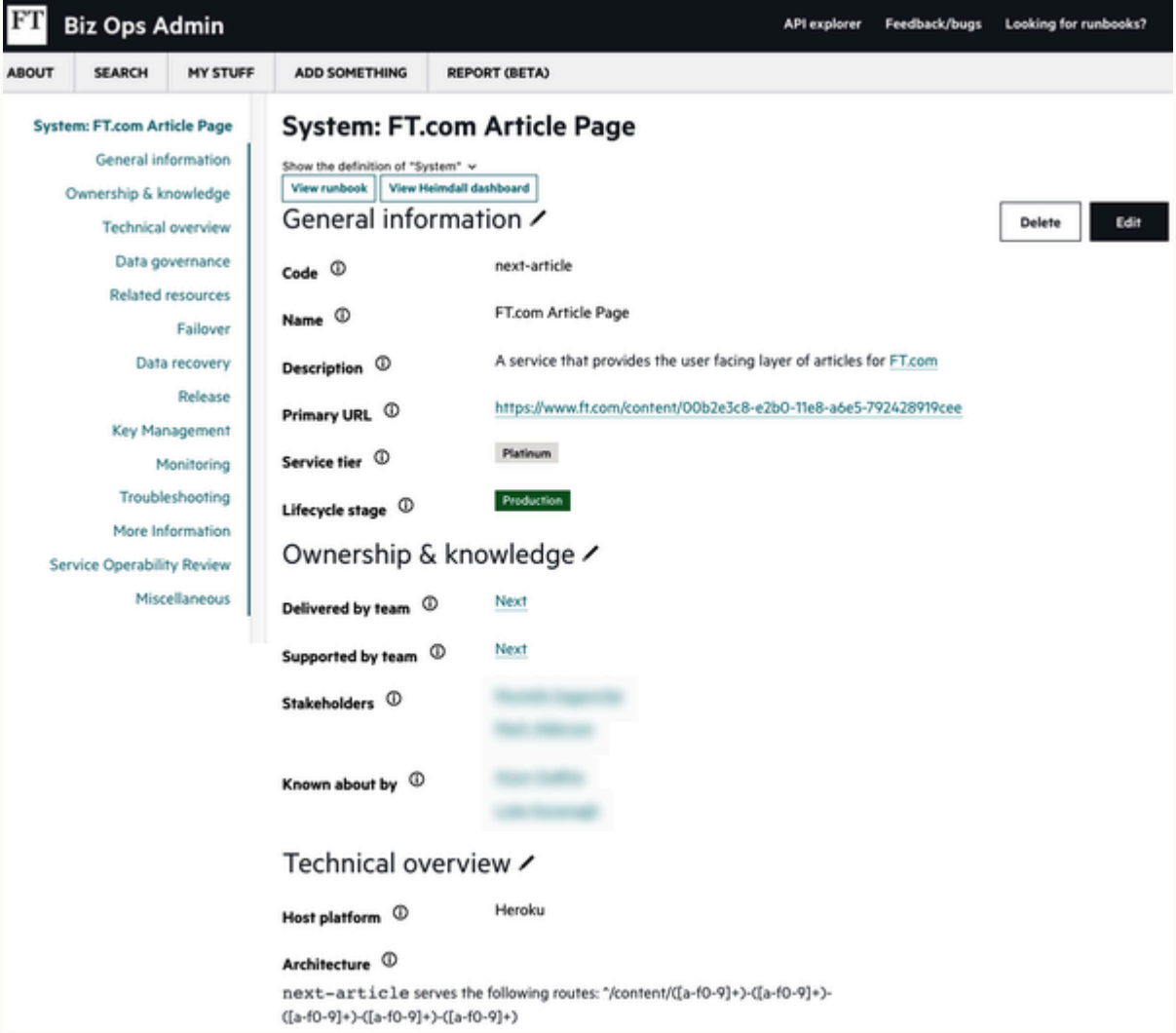


Figure 5-8. The Financial Times Biz Ops tool, which collates information about its microservices

The Biz Ops tool goes further than most of the similar tools I have seen, however. The tool calculates what it calls a System Operability Score, as shown in Figure 5-9. The idea is that there are certain things that services and their teams should do to ensure the services can be easily operated. This can range from making sure the teams have provided the correct information in the registry to ensuring the services have proper health checks. The System Operability Score, once calculated, allows teams to see at a glance if there are things that need to be fixed.

This is a growing space. In the open source world, Spotify's Backstage tool offers a mechanism for building a service catalog like Biz Ops, with a plug-in model to allow for sophisticated additions, such as being able to trigger the creation of a new microservice or pulling in live information from a Kubernetes cluster. Ambassador's own Service Catalog is more narrowly focused on visibility of services in Kubernetes, which means it might not have as much general appeal as something like the FT's Biz Ops, but it's nonetheless good to see some new takes on this idea that are more generally available.

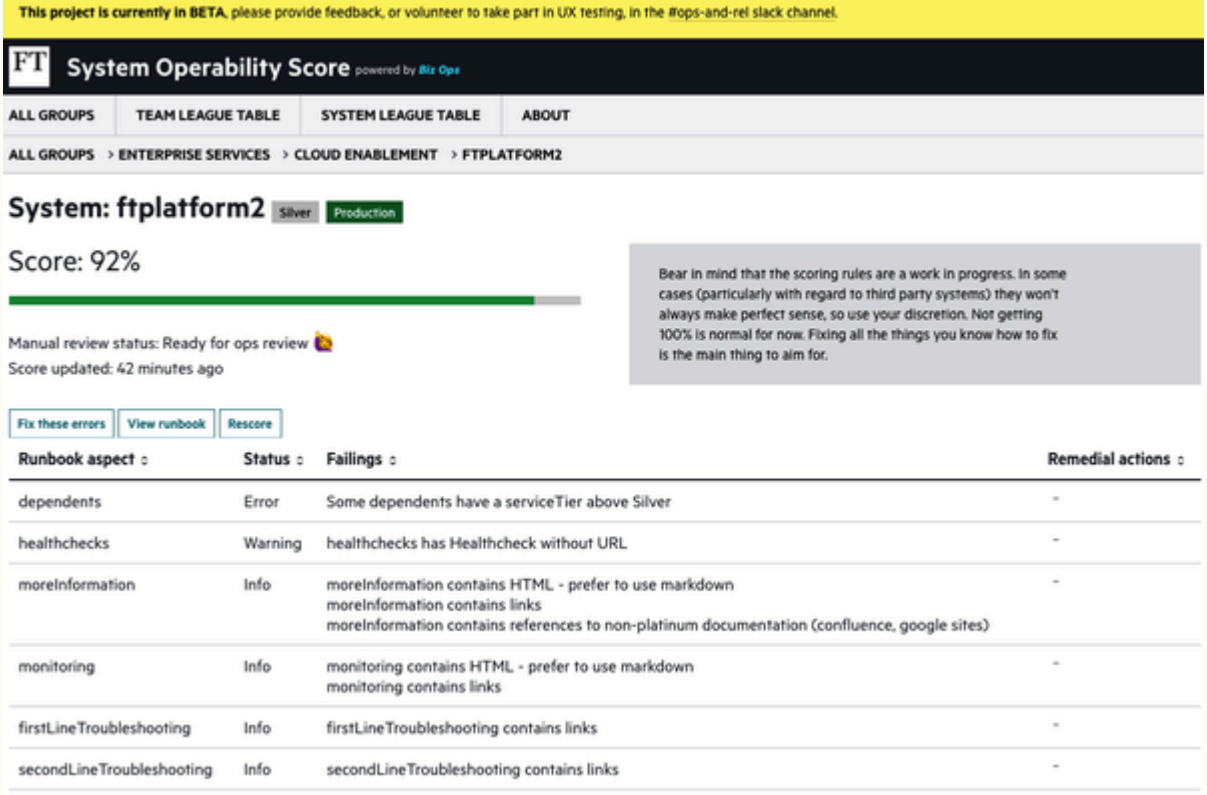


Figure 5-9. An example of the Service Operability Score for a microservice at the Financial Times

Summary

So we've covered a lot of ground in this chapter—let's break some of it down:

- To begin with, ensure that the problem you are trying to solve guides your technology choice. Based on your context and your preferred communication style, select the technology that is most appropriate for you—don't fall into the trap of picking the technology first. The summary of styles of inter-microservice communication, first introduced in Chapter 4 and shown again in Figure 5-10, can help guide your decision making, but just following this model isn't a replacement for sitting down and thinking about your own situation.

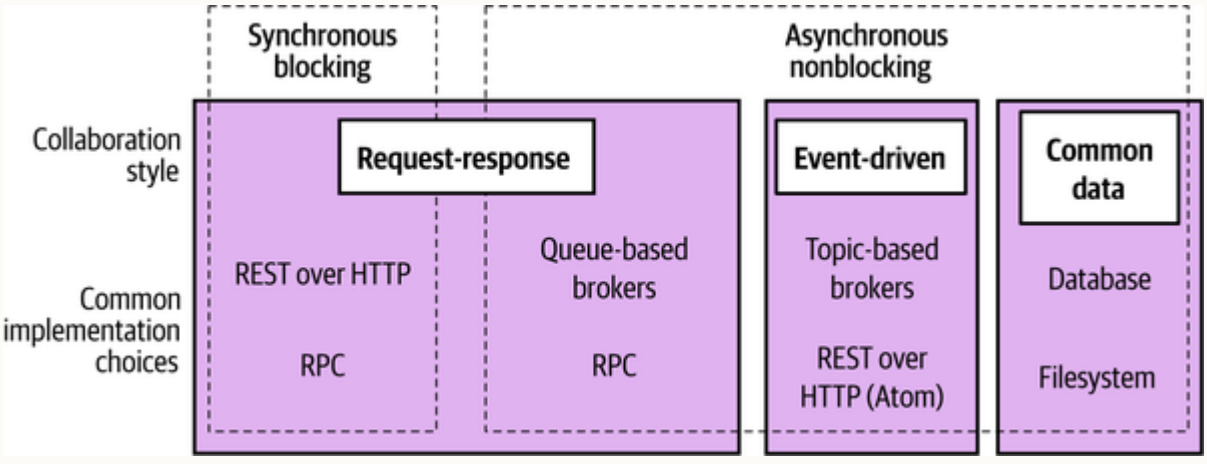


Figure 5-10. Different styles of inter-microservice communication, along with example implementing technologies

- Whatever choice you make, consider the use of schemas, in part to help make your contracts more explicit but also to help catch accidental breaking changes.
- Where possible, strive to make changes that are backward compatible to ensure that independent deployability remains a possibility.
- If you do have to make backward-incompatible changes, find a way to allow consumers time to upgrade to avoid lockstep deployments.
- Think about what you can do to help surface information about your endpoints to humans—consider the use of humane registries and the like to help make sense of the chaos.

We've looked at how we can implement a call between two microservices, but what happens when we need to coordinate operations between multiple microservices? That will be the focus of our next chapter.

1 Martin Fowler explores this in more detail in the context of schemaless data storage.

2 Note that there are actually three different tools in this space with the same name! The openapi-diff tool at *https://github.com/Azure/openapi-diff* seems to get closest to a tool that actually passes or fails compatibility.